

RTL Design Sherpa

APB UART 16550 Micro-Architecture Specification 1.0

January 4, 2026

Table of Contents

1 Uart 16550 Mas Index....20

2 Retro Legacy Blocks - Product Requirements Document....21

2.1 1. Overview....21

2.1.1 1.1 Purpose....21

2.1.2 1.2 Design Philosophy....21

2.1.3 1.3 Target Applications....21

2.2 2. Implemented Blocks....22

2.2.1 2.1 HPET - High Precision Event Timer....22

2.3 3. Planned Blocks....23

2.3.1 3.1 8259 - Programmable Interrupt Controller (PIC)....23

2.3.2 3.2 8254 - Programmable Interval Timer (PIT)....24

2.3.3 3.3 GPIO - General Purpose I/O....24

2.3.4 3.4 RTC - Real-Time Clock....24

2.3.5 3.5 SMBus Controller....25

2.3.6 3.6 UART - Universal Asynchronous Receiver/Transmitter....25

2.3.7 3.7 SPI Controller....25

2.3.8 3.8 I2C Controller....25

2.3.9 3.9 Watchdog Timer....25

2.3.10 3.10 Power Management / ACPI Controller....26

2.3.11 3.11 IOAPIC - I/O Advanced Programmable Interrupt Controller....26

2.3.12 3.12 Interconnect ID / Version Registers....26

2.4 4. Integration and Wrapper Goals....27

2.4.1 4.1 Individual Block Integration....27
2.4.2 4.2 RLB Wrapper Architecture....27
2.5 5. Design Standards....30
2.5.1 5.1 Reset Handling....30
2.5.2 5.2 Register Generation....30
2.5.3 5.3 Testbench Architecture....30
2.5.4 5.4 FPGA Synthesis Attributes....31
2.5.5 5.5 Documentation Requirements....31
2.6 6. Quality Metrics....31
2.6.1 6.1 Production Readiness Criteria....31
2.6.2 6.2 Current Status....32
2.7 7. Development Roadmap....33
2.7.1 7.1 Phase 1: Foundation (Complete)....33
2.7.2 7.2 Phase 2: Core Peripherals (Next 6-9 Months)....33
2.7.3 7.3 Phase 3: Advanced Peripherals (9-15 Months)....33
2.7.4 7.4 Phase 4: System Integration (15+ Months)....33
2.8 8. References....34
2.8.1 8.1 External Standards....34
2.8.2 8.2 Internal Documentation....34
2.8.3 8.3 Block-Specific Documentation....34
2.9 9. Success Criteria....34
2.9.1 9.1 Individual Block Success....34
2.9.2 9.2 Collection Success....34
2.9.3 9.3 Long-Term Vision....35

3 PeakRDL HPET Integration - Final Status....36
3.1 Milestone: COMPLETE (5/6 configs fully passing)....36
3.1.1 Test Results Summary....36
3.2 Root Cause Found & Fixed....36
3.3 Core Functionality Validated....37
3.4 Files Modified....37
3.4.1 RTL Changes:....37
3.4.2 Test Changes:....37
3.4.3 Documentation:....38
3.5 Remaining Work (Minor)....38
3.5.1 8-Timer Non-CDC All Timers Stress Test....38
3.6 Milestone Achievement....38
3.7 Recommended Next Steps....38
3.8 Test Execution Summary....38
3.9 Git Status....39
4 APB UART 16550 — Overview....40
4.1 Overview....40
4.1.1 Key Features....40
4.1.2 Applications....41
4.2 Parameters....41
4.3 Functional Description....42
4.3.1 Figure 1.1: UART 16550 Block Diagram....42
4.3.2 Register Summary....42
4.4 Design Notes....43

4.4.1 Compatibility....43
4.5 Navigation....43
5 APB UART 16550 — Architecture....44
5.1 Overview....44
5.2 Functional Description....44
5.2.1 Figure 1.2: UART Architecture....44
5.2.2 Module Hierarchy....44
5.2.3 Data Flow....46
5.2.4 Baud Rate Generation....47
5.2.5 FIFO Organization....48
5.3 Design Notes....49
5.3.1 Resource Estimates....49
5.4 Navigation....49
6 APB UART 16550 — Clocks and Reset....50
6.1 Overview....50
6.2 Ports....50
6.2.1 Clock Signals....50
6.2.2 Reset Signals....50
6.3 Functional Description....51
6.3.1 Reset Behavior....51
6.3.2 Clock Domain Crossing....52
6.4 Timing....52
6.4.1 Baud Rate Considerations....52
6.4.2 Timing Constraints....53

6.5 Navigation....53
7 APB UART 16550 — Acronyms and Terminology....54
7.1 Overview....54
7.2 Design Notes....54
7.2.1 Protocol Acronyms....54
7.2.2 Register Acronyms....54
7.2.3 Signal Acronyms....55
7.2.4 FIFO Terms....55
7.2.5 Data Format Terms....56
7.2.6 Error Terms....56
7.2.7 Timing Terms....56
7.3 Navigation....57
8 APB UART 16550 — References....58
8.1 Overview....58
8.2 References....58
8.2.1 Internal Documentation....58
8.2.2 External References....58
8.2.3 Design References....59
8.3 Navigation....59
9 APB UART 16550 — Block Descriptions....60
9.1 Overview....60
9.2 Functional Description....60
9.2.1 Module Hierarchy....60
9.2.2 Block Summary....60

9.3 Related Modules..... 61
9.3.1 APB Interface..... 61
9.3.2 Register File..... 61
9.3.3 TX Engine..... 61
9.3.4 RX Engine..... 61
9.3.5 Baud Generator..... 61
9.3.6 FIFO Subsystem..... 61
9.4 Navigation..... 61
10 APB UART 16550 — APB Interface Block..... 63
10.1 Overview..... 63
10.2 Ports..... 63
10.2.1 APB Slave Interface..... 63
10.3 Functional Description..... 64
10.3.1 Figure 2.1: APB Interface Block..... 64
10.3.2 Address Decoding..... 64
10.3.3 Operation..... 65
10.4 Design Notes..... 65
10.4.1 Implementation Notes..... 65
10.5 Navigation..... 66
11 APB UART 16550 — Register File Block..... 67
11.1 Overview..... 67
11.2 Ports..... 67
11.2.1 Software-to-Hardware (reg2hw)..... 67
11.2.2 Hardware-to-Software (hw2reg)..... 67

11.3 Functional Description..... 68
11.3.1 Figure 2.2: Register File Block.....68
11.3.2 Register Organization.....68
11.3.3 Register Access Types.....69
11.4 Design Notes..... 69
11.4.1 Implementation Notes.....69
11.5 Navigation.....69
12 APB UART 16550 — TX Engine Block.....70
12.1 Overview..... 70
12.2 Functional Description..... 70
12.2.1 Figure 2.3: TX Engine Block.....70
12.2.2 Data Path..... 70
12.2.3 TX FIFO.....70
12.2.4 TX Serializer.....71
12.2.5 Flow Control..... 72
12.2.6 Break Generation.....73
12.3 Timing..... 73
12.3.1 Bit Timing.....73
12.3.2 Frame Timing Example (8N1 at 115200).....73
12.4 Waveforms..... 73
12.4.1 Waveform 2.1: TX Byte Transmission.....73
12.4.2 Waveform 2.2: Baud Rate Generation.....74
12.4.3 Waveform 2.3: Loopback Mode.....74
12.5 Navigation.....74

13 APB UART 16550 — RX Engine Block....75
13.1 Overview....75
13.2 Functional Description....75
13.2.1 Figure 2.4: RX Engine Block....75
13.2.2 Data Path....76
13.2.3 Input Synchronizer....76
13.2.4 Start Bit Detection....76
13.2.5 RX Deserializer....77
13.2.6 RX FIFO....78
13.2.7 Error Detection....78
13.2.8 Timeout Detection....79
13.3 Waveforms....79
13.3.1 Waveform 2.4: RX Byte Reception....79
13.4 Navigation....80
14 APB UART 16550 — Baud Generator Block....81
14.1 Overview....81
14.2 Functional Description....81
14.2.1 Figure 2.5: Baud Generator Block....81
14.2.2 Operation....82
14.2.3 Divisor Calculation....82
14.2.4 Divisor Latch Registers....83
14.3 Usage Example....83
14.3.1 Programming Sequence....83
14.4 Design Notes....83

14.4.1 Special Cases....83
14.5 Navigation....84
15 APB UART 16550 — FIFO Subsystem....85
15.1 Overview....85
15.2 Functional Description....85
15.2.1 Figure 2.6: FIFO Block....85
15.2.2 FIFO Configuration....85
15.2.3 TX FIFO....86
15.2.4 RX FIFO....87
15.2.5 FIFO vs Non-FIFO Mode....87
15.2.6 Error Handling....88
15.2.7 FIFO Reset....88
15.3 Waveforms....89
15.3.1 Waveform 2.5: FIFO Trigger Timing....89
15.4 Navigation....89
16 APB UART 16550 — Interfaces....90
16.1 Overview....90
16.2 Ports....90
16.2.1 External Interfaces....90
16.3 Functional Description....91
16.3.1 Figure 3.1: UART Interface Summary....91
16.4 Navigation....91
16.4.1 Chapter Contents....91
17 APB UART 16550 — APB Slave Interface....93

17.1 Overview..... 93
17.2 Ports..... 93
17.2.1 APB Slave Signals..... 93
17.3 Functional Description..... 93
17.3.1 Address Map..... 93
17.3.2 Protocol Compliance..... 94
17.3.3 Register Access..... 95
17.4 Timing..... 95
17.4.1 Access Timing..... 95
17.5 Navigation..... 95
18 APB UART 16550 — Serial Interface..... 97
18.1 Overview..... 97
18.2 Ports..... 97
18.2.1 Signal Description..... 97
18.2.2 Electrical Interface..... 97
18.3 Functional Description..... 97
18.3.1 TXD (Transmit Data)..... 97
18.3.2 RXD (Receive Data)..... 98
18.3.3 Data Formats..... 99
18.3.4 Break Condition..... 99
18.4 Waveforms..... 99
18.4.1 Waveform 3.1: Line Status Error Detection..... 99
18.5 Navigation..... 100
19 APB UART 16550 — Modem Interface..... 101

19.1 Overview....101
19.2 Ports....101
19.2.1 Modem Control Outputs (Active Low)....101
19.2.2 Modem Status Inputs (Active Low)....101
19.3 Functional Description....101
19.3.1 Modem Control Register (MCR)....101
19.3.2 Modem Status Register (MSR)....102
19.3.3 Hardware Flow Control....103
19.3.4 Figure 3.2: RTS/CTS Flow Control Connections....103
19.3.5 Loopback Mode....104
19.3.6 Input Synchronization....104
19.3.7 Interrupt Generation....104
19.4 Waveforms....104
19.4.1 Waveform 3.2: Modem Status Change Detection....104
19.5 Navigation....105
20 APB UART 16550 — Interrupt Interface....106
20.1 Overview....106
20.2 Ports....106
20.2.1 Signal Description....106
20.3 Functional Description....106
20.3.1 Interrupt Sources....106
20.3.2 Interrupt Enable Register (IER)....107
20.3.3 Interrupt Identification Register (IIR)....107
20.3.4 Interrupt Conditions....108

20.4 Timing....108
20.4.1 Interrupt Timing....108
20.5 Usage Example....109
20.5.1 Software Handling....109
20.6 Navigation....109
21 APB UART 16550 — System Interface....110
21.1 Overview....110
21.2 Ports....110
21.2.1 Clock Signals....110
21.2.2 Reset Signals....110
21.3 Functional Description....111
21.3.1 Reset Behavior....111
21.4 Timing....112
21.4.1 Reset Sequence....112
21.4.2 Figure 3.3: Reset Sequence....112
21.5 Design Notes....112
21.5.1 Power Management....112
21.5.2 External Connections....113
21.5.3 Figure 3.4: UART 16550 in a System....113
21.6 Navigation....114
22 APB UART 16550 — Programming Model....115
22.1 Overview....115
22.1.1 Register Summary....115
22.2 Usage Example....115

22.2.1 Quick Start 115
22.3 Navigation 116
22.3.1 Chapter Contents 116
23 APB UART 16550 — Initialization 118
23.1 Overview 118
23.2 Functional Description 118
23.2.1 Baud Rate Calculation 118
23.3 Usage Example 118
23.3.1 Basic Initialization Sequence 118
23.3.2 Complete Initialization Example 120
23.3.3 Flow Control Setup (manual) 120
23.3.4 Loopback Mode Setup 121
23.4 Navigation 121
24 APB UART 16550 — Data Transfer 122
24.1 Overview 122
24.2 Usage Example 122
24.2.1 Transmitting Data 122
24.2.2 Receiving Data 123
24.2.3 Error Handling 124
24.2.4 FIFO Management 125
24.2.5 Bulk Transfer 125
24.3 Navigation 126
25 APB UART 16550 — Interrupt Handling 127
25.1 Overview 127

25.2 Functional Description....127
25.2.1 Interrupt Identification Register (IIR)....127
25.3 Usage Example....128
25.3.1 Interrupt Enable Register (IER)....128
25.3.2 Complete ISR Example....128
25.3.3 Individual Interrupt Handlers....129
25.3.4 Disabling/Enabling Interrupts....131
25.4 Design Notes....131
25.4.1 Interrupt Latency Considerations....131
25.5 Navigation....132
26 APB UART 16550 — Programming Examples....133
26.1 Overview....133
26.2 Usage Example....133
26.2.1 Debug Console....133
26.2.2 Ring Buffer Implementation....134
26.2.3 Command Line Interface....136
26.2.4 Loopback Test....137
26.2.5 Flow Control (manual)....138
26.3 Navigation....138
27 APB UART 16550 — Register Map....139
27.1 Overview....139
27.2 Functional Description....139
27.2.1 Register Summary....139
27.2.2 RBR - Receiver Buffer Register (0x00, Read)....140

27.2.3 THR - Transmitter Holding Register (0x00, Write).....140
27.2.4 IER - Interrupt Enable Register (0x04, R/W).....140
27.2.5 IIR - Interrupt Identification Register (0x08, Read Only).....141
27.2.6 FCR - FIFO Control Register (0x0C, R/W).....142
27.2.7 LCR - Line Control Register (0x10, R/W).....143
27.2.8 MCR - Modem Control Register (0x14, R/W).....144
27.2.9 LSR - Line Status Register (0x18, Read; error bits clear on read).....144
27.2.10 MSR - Modem Status Register (0x1C, Read; delta bits clear on read) 145
27.2.11 SCR - Scratch Register (0x20, R/W).....146
27.2.12 DLL - Divisor Latch LSB (0x24, R/W).....146
27.2.13 DLM - Divisor Latch MSB (0x28, R/W).....146
27.2.14 Address Calculation.....146
27.3 Implemented for RLB TASK-012 (was RLB-013).....147

List of Figures

Figure 1.1: UART 16550 Block Diagram....42
Figure 1.2: UART Architecture....44
Figure 2.1: APB Interface Block....63
Figure 2.2: Register File Block....67
Figure 2.3: TX Engine Block....69
Figure 2.4: RX Engine Block....74
Figure 2.5: Baud Generator Block....80
Figure 2.6: FIFO Block....84
Figure 3.1: UART Interface Summary....90
Figure 3.2: RTS/CTS Flow Control Connections....102
Figure 3.3: Reset Sequence....111
Figure 3.4: UART 16550 in a System....112

List of Tables

No tables in this document.

List of Waveforms

Waveform 2.1: TX Byte Transmission....72
Waveform 2.2: Baud Rate Generation....73
Waveform 2.3: Loopback Mode....73
Waveform 2.4: RX Byte Reception....78
Waveform 2.5: FIFO Trigger Timing....87
Waveform 3.1: Line Status Error Detection....98
Waveform 3.2: Modem Status Change Detection....103

1 Uart 16550 Mas Index

Generated: 2026-10-20

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

2 Retro Legacy Blocks - Product Requirements Document

Component: Retro Legacy Blocks (RLB) - Production-Quality Legacy Peripherals **Version:** 1.0
Status: Active Development - HPET Production Ready **Last Updated:** 2025-10-29

2.1 1. Overview

2.1.1 1.1 Purpose

The Retro Legacy Blocks (RLB) component provides production-quality implementations of legacy peripheral blocks based on proven peripheral designs. These blocks are designed to be reusable, well-tested, and suitable for both FPGA and ASIC implementation.

2.1.2 1.2 Design Philosophy

“Retro” - Proven Architectures: - Implements time-tested peripheral designs from successful platforms - Focuses on simplicity, reliability, and well-understood behavior - Prioritizes production-readiness over experimental features

“Legacy” - Time-Tested Interfaces: - Based on proven peripheral interface specifications - Suitable for systems requiring retro-compatible peripheral compatibility - APB-based interface for easy integration

“Blocks” - Modular Collection: - Each peripheral is independent and self-contained - Clear separation between different blocks (rtl/hpet/, rtl/gpio/, etc.) - Can be used individually or wrapped into integrated subsystem

2.1.3 1.3 Target Applications

- Retro-compatible platform compatibility layers
- Embedded systems requiring legacy peripheral interfaces
- FPGA-based system emulation
- Educational platforms demonstrating classic peripheral designs
- Mixed-vintage SoC integration (modern + legacy interfaces)

2.2 2. Implemented Blocks

2.2.1 2.1 HPET - High Precision Event Timer

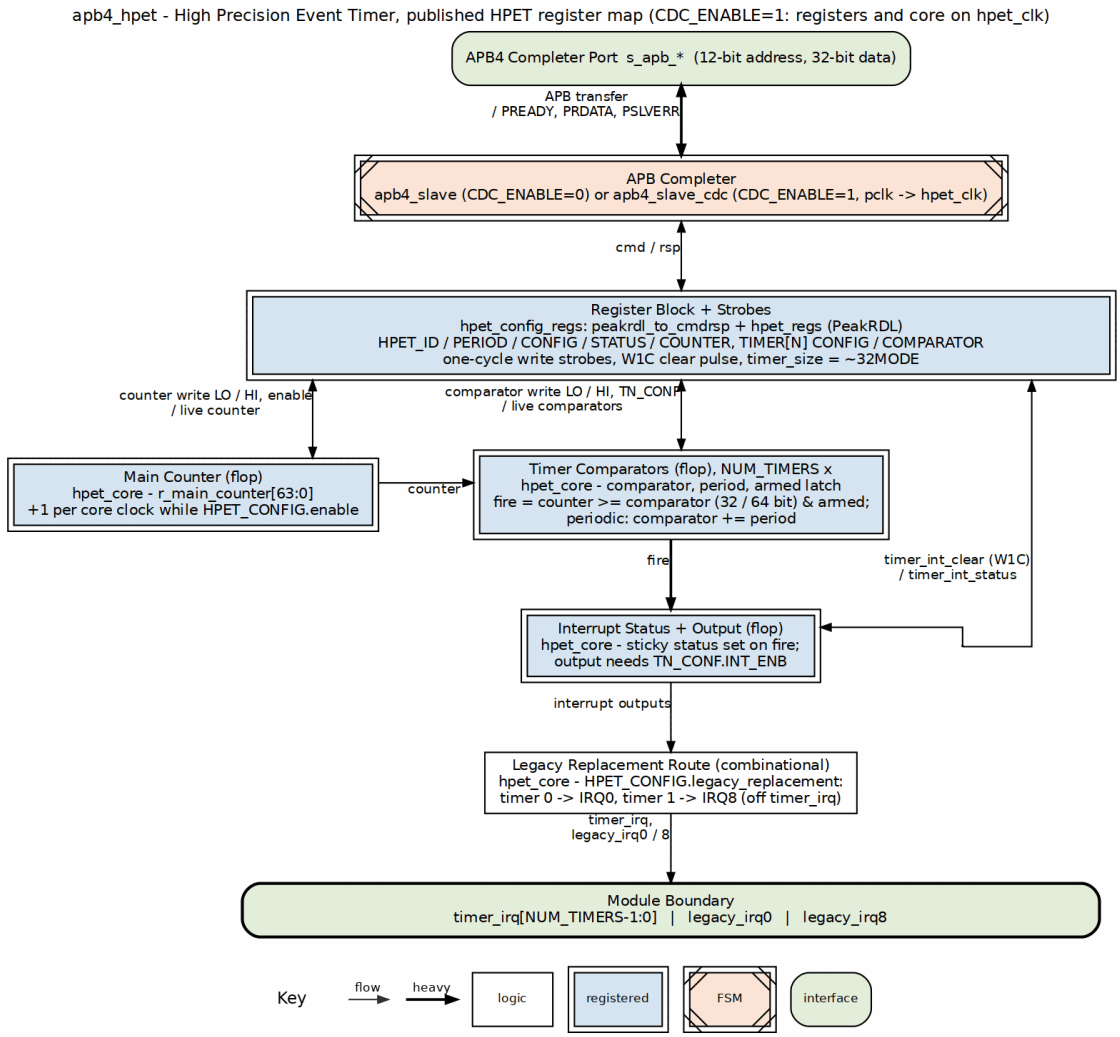
Status: Production Ready (5/6 configurations 100% passing) **RTL Location:** `rtl/hpet/` **Documentation:** `docs/hpet_mas/`

Key Features: - Configurable timer count: 2, 3, or 8 independent timers - 64-bit main counter for high-resolution timestamps - 64-bit comparators per timer - Operating modes: One-shot and periodic - Clock domain crossing: Optional CDC for timer/APB clock independence - APB4 interface: Standard AMBA APB protocol - PeakRDL integration: Register map generated from SystemRDL specification

Applications: - System tick generation - Real-time OS scheduling - Precise event timing - Performance profiling - Watchdog timers - Multi-rate timing domains

Test Coverage: - 6 configurations tested (2/3/8 timers, CDC on/off) - 5/6 configurations at 100% pass rate - 1 configuration at 92% (minor stress test timeout) - 12 test cases per configuration (gate/func/full)

Architecture:



Design Highlights: - Reset macro standardization (FPGA-friendly) - Per-timer data buses prevent corruption - Edge-triggered register write strobes (not level) - W1C status register for interrupt clearing - Optional asynchronous clock domains with handshake CDC

See: `docs/hpet_mas/hpet_mas_index.md` for complete HPET specification

2.3 3. Planned Blocks

2.3.1 3.1 8259 - Programmable Interrupt Controller (PIC)

Status: Implemented (see dv/tests and the block MAS; per-block defects tracked in known_issues/)
Priority: High **Address:** `0xFEC0_1000` - `0xFEC0_1FFF` (4KB window, master of the cascaded pair)

Planned Features: - 8259A-compatible register interface - 8 interrupt request (IRQ) inputs - Cascadable (master/slave configuration) - Priority resolver (fixed and rotating priority) - Edge and level triggered modes - Interrupt mask register - End-of-Interrupt (EOI) handling - APB register interface

Applications: - Legacy interrupt management - PC-compatible systems - Hardware interrupt aggregation - Priority-based interrupt handling - Cascaded multi-level interrupt systems

2.3.2 3.2 8254 - Programmable Interval Timer (PIT)

Status: Implemented (see dv/tests and the block MAS; per-block defects tracked in known_issues/) **Priority:** High **Address:** `0xFEC0_2000 - 0xFEC0_2FFF` (4KB window)

Planned Features: - 8254-compatible register interface - 3 independent 16-bit counters - 6 programmable counter modes - Binary and BCD counting - Read-back command - Configurable clock input - Interrupt/output generation per counter - APB register interface

Counter Modes: - Mode 0: Interrupt on terminal count - Mode 1: Hardware retriggerable one-shot - Mode 2: Rate generator - Mode 3: Square wave mode - Mode 4: Software triggered strobe - Mode 5: Hardware triggered strobe

Applications: - System tick generation - Periodic timer interrupts - Square wave generation - Event counting - Legacy PC timer compatibility

2.3.3 3.3 GPIO - General Purpose I/O

Status: Implemented (see dv/tests and the block MAS; per-block defects tracked in known_issues/) **Priority:** Medium **Address:** `0xFEC0_7000 - 0xFEC0_7FFF` (4KB window)

Planned Features: - Configurable pin count (8, 16, 32 pins) - Per-pin direction control (input/output/bidirectional) - Input debouncing logic - Interrupt generation (rising/falling/both edges, level) - Output drive strength configuration - Pull-up/pull-down control - APB register interface

Applications: - LED control - Button inputs - Hardware control signals - Chip-select generation - Status monitoring

2.3.4 3.4 RTC - Real-Time Clock

Status: Implemented (apb4_rtc + MAS; see docs/rtc_mas – binary 24-hour timekeeping validated; BCD/12-hour modes and the production-mode time-set path are broken, tracked in issue #56) **Priority:** Medium **Effort:** 3-4 weeks **Address:** `0xFEC0_3000 - 0xFEC0_3FFF` (4KB window)

Planned Features: - 32.768 kHz clock input (typical RTC crystal frequency) - Seconds, minutes, hours, day, month, year tracking - Alarm functionality - Battery backup support (power domain considerations) - 24-hour or 12-hour (AM/PM) mode - Leap year handling - APB register interface

Applications: - System time-of-day tracking - Wake-on-alarm functionality - Timestamp generation - Power-aware applications

2.3.5 3.5 SMBus Controller

Status: Implemented (see dv/tests and the block MAS; per-block defects tracked in known_issues/) **Priority:** Medium **Address:** `0xFEC0_4000 - 0xFEC0_4FFF` (4KB window)

Planned Features: - SMBus 2.0 compliance - Master and slave modes - Clock stretching support - Packet Error Checking (PEC) - Alert response address - Configurable clock speed - APB register interface

Applications: - System management bus communication - Sensor interfaces (temperature, voltage) - EEPROM access - Battery management - Fan control

2.3.6 3.6 UART - Universal Asynchronous Receiver/Transmitter

Status: Implemented (apb4_uart_16550 + MAS; see docs/uart_16550_mas – note the shipped block has 1/2 stop bits only and no automatic flow control, and issue #60 tracks its RTL defects) **Priority:** Medium **Address:** `0xFEC0_8000 - 0xFEC0_8FFF` (4KB window)

Planned Features: - 16550-compatible register interface - Configurable baud rate generation - 5/6/7/8 data bits - Parity: none, even, odd, mark, space - Stop bits: 1, 1.5, 2 - Hardware flow control (RTS/CTS) - FIFO buffers (16-byte TX/RX) - Interrupt generation

Applications: - Debug console - Serial communication - Modem interfaces - Legacy peripheral communication

2.3.7 3.7 SPI Controller

Status: Planned **Priority:** Low **Effort:** 5-6 weeks **Address:** TBD (not in primary ILB address map)

Planned Features: - Master mode (initially; slave mode future) - Configurable clock polarity and phase (CPOL/CPHA) - Multiple chip selects - Configurable word size (8/16/32 bits) - TX/RX FIFOs - DMA support (future) - APB register interface

Applications: - Flash memory access - ADC/DAC interfaces - Display controllers - SD card communication

2.3.8 3.8 I2C Controller

Status: Planned **Priority:** Low **Effort:** 5-7 weeks **Address:** TBD (not in primary ILB address map)

Planned Features: - I2C standard (100 kHz), fast (400 kHz), fast-plus (1 MHz) modes - Multi-master arbitration - 7-bit and 10-bit addressing - Clock stretching - General call support - APB register interface

Applications: - Sensor interfaces - EEPROM access - Multi-chip communication - System configuration

2.3.9 3.9 Watchdog Timer

Status: Planned **Priority:** Low **Effort:** 2-3 weeks **Address:** TBD (not in primary ILB address map)

Planned Features: - Configurable timeout period - Countdown counter with reload - Reset generation on timeout - Lock mechanism to prevent accidental disable - Interrupt before reset (optional warning) - APB register interface

Applications: - System fault recovery - Software hang detection - Periodic system reset - Safety-critical applications

2.3.10 3.10 Power Management / ACPI Controller

Status: Implemented (see dv/tests and the block MAS; per-block defects tracked in known_issues/) **Priority:** Medium **Address:** `0xFEC0_5000 - 0xFEC0_5FFF` (4KB window)

Planned Features: - Clock gating control per block - Power domain sequencing - Reset generation and distribution - Wake event handling - Sleep/idle mode control - ACPI-compatible registers - APB register interface

Applications: - Low-power system design - Battery-powered devices - Dynamic power management - Thermal management - OS power management interface

2.3.11 3.11 IOAPIC - I/O Advanced Programmable Interrupt Controller

Status: Implemented (see dv/tests and the block MAS; per-block defects tracked in known_issues/) **Priority:** Medium **Address:** `0xFEC0_6000 - 0xFEC0_6FFF` (4KB window)

Planned Features: - I/O APIC CSR model (register-based interface) - Multiple interrupt inputs (24+) - Programmable interrupt routing - Edge and level triggered modes - Priority-based arbitration - Interrupt masking per input - APB register interface for configuration

Applications: - Advanced interrupt routing - Multi-processor interrupt distribution - Flexible interrupt mapping - Legacy IRQ redirection - PC-compatible systems

2.3.12 3.12 Interconnect ID / Version Registers

Status: Planned – NOT present in the shipped design **Priority:** Low **Effort:** 1-2 weeks **Address:** unallocated. The shipped decoder is the generated `apbx_xbar_1to10`, whose ten windows (`0x0000`-`0x9FFF`) are fully assigned, so adding this block needs a window assignment AND a crossbar regeneration – it cannot simply occupy `0xF000` as earlier drafts of this document assumed.

Planned Features: - Vendor ID register - Device ID register - Revision ID register - Block presence/capability bits - Configuration status registers - Debug/diagnostic registers - APB register interface

Applications: - Software block discovery - Version checking - Feature detection - Debug and diagnostics - Platform identification

---

2.4 4. Integration and Wrapper Goals

2.4.1 4.1 Individual Block Integration

Each block is designed to be used standalone:

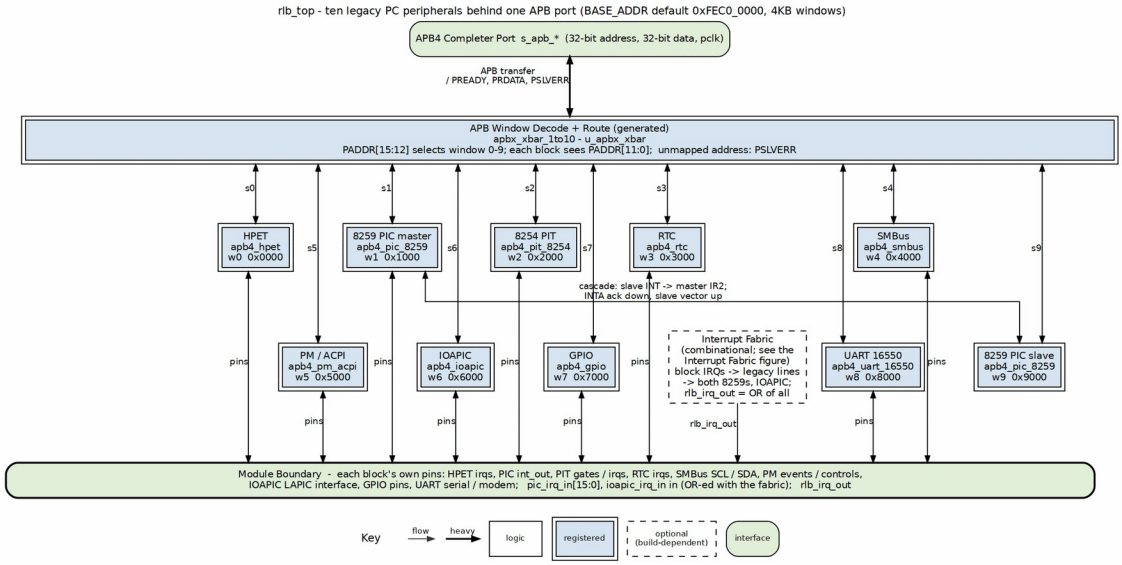
Example - HPET Integration:

```
apb4_hpet #(
    .NUM_TIMERS(3),
    .VENDOR_ID(16'h4747),
    .REVISION_ID(16'h0001),
    .CDC_ENABLE(0)
) u_hpet (
    .pclk           (apb_clk),
    .presetn        (apb_rst_n),
    // APB interface -- the ports are s_apb_P*, not bare p*
    .s_apb_PSEL     (psel_hpet),
    .s_apb_PENABLE  (penable),
    .s_apb_PREADY   (pready_hpet),
    .s_apb_PADDR    (paddr[11:0]),
    .s_apb_PWRITE   (pwrite),
    .s_apb_PWDATA   (pwdata),
    .s_apb_PSTRB    (pstrb),
    .s_apb_PPROT    (pprot),
    .s_apb_PRDATA   (prdata_hpet),
    .s_apb_PSLVERR  (pslverr_hpet),
    // HPET-specific
    .hpet_clk       (timer_clk),
    .hpet_resetn    (timer_rst_n),
    .timer_irq    (timer_irq[2:0])
);
```

2.4.2 4.2 RLB Wrapper Architecture

Goal: Create top-level wrapper combining multiple legacy blocks into unified retro-compatible subsystem.

System Architecture:



Address Map:

Base address: `0xFEC0_0000` – the `BASE_ADDR` parameter default in `rlb_top.sv`, and the conventional location for this class of block. It is a PARAMETER: an integrator may relocate the whole window set. Window size: 4KB per block (clean power-of-2 decode)

Address Range	Block	Size	Function
`0xFEC0_0000 - 0xFEC0_0FFF`	HPET	4KB	High Precision Event Timer
`0xFEC0_1000 - 0xFEC0_1FFF`	8259 PIC (master)	4KB	Programmable Interrupt Controller
`0xFEC0_2000 - 0xFEC0_2FFF`	8254 PIT	4KB	Programmable Interval Timer
`0xFEC0_3000 - 0xFEC0_3FFF`	RTC	4KB	Real-Time Clock
`0xFEC0_4000 - 0xFEC0_4FFF`	SMBus	4KB	SMBus Host Controller
`0xFEC0_5000 - 0xFEC0_5FFF`	PM/ACPI	4KB	Power Management / ACPI Registers
`0xFEC0_6000 - 0xFEC0_6FFF`	IOAPIC	4KB	I/O Advanced PIC (CSR model)
`0xFEC0_7000 - 0xFEC0_7FFF`	GPIO	4KB	General Purpose

Address Range	Block	Size	Function
			I/O
0xFEC0_8000 - 0xFEC0_8FFF	UART 16550	4KB	Serial port
0xFEC0_9000 - 0xFEC0_9FFF	8259 PIC (slave)	4KB	Cascade slave of the PC/AT pair
All other addresses	decode-miss agent	-	Accepted and answered locally with PSLVERR

The decoder is the GENERATED `apbx_xbar_1to10` (one master, ten slaves), so the window count and the base both come from the generator invocation recorded in `rlb_top.sv`. There is no window above `0x9FFF`.

Decoder Implementation:

```
// Address decode logic (simplified)
localparam BASE_ADDR = 32'hFEC0_0000;
localparam BLOCK_SIZE = 12;  // 4KB = 2^12

logic [3:0] block_sel;
assign block_sel = paddr[15:12];  // Extract window number

always_comb begin
    psel_hpet      = (block_sel == 4'h0) & psel;  // 0xFEC0_0xxx
    psel_pic8259   = (block_sel == 4'h1) & psel;  // 0xFEC0_1xxx
    psel_pit8254   = (block_sel == 4'h2) & psel;  // 0xFEC0_2xxx
    psel_rtc       = (block_sel == 4'h3) & psel;  // 0xFEC0_3xxx
    psel_smbus     = (block_sel == 4'h4) & psel;  // 0xFEC0_4xxx
    psel_pm        = (block_sel == 4'h5) & psel;  // 0xFEC0_5xxx
    psel_ioapic    = (block_sel == 4'h6) & psel;  // 0xFEC0_6xxx
    psel_gpio      = (block_sel == 4'h7) & psel;  // 0xFEC0_7xxx
    psel_uart      = (block_sel == 4'h8) & psel;  // 0xFEC0_8xxx
    psel_pic_slave = (block_sel == 4'h9) & psel;  // 0xFEC0_9xxx
    psel_error     = !(|{psel_hpet, psel_pic8259, psel_pit8254,
                         psel_rtc, psel_smbus, psel_pm, psel_ioapic,
                         psel_gpio, psel_uart, psel_pic_slave}) &
psel;
end
```

Interface: - **Single APB slave port** at base address `0xFEC0_0000` (the `BASE_ADDR` parameter) - **Aggregated interrupt output** combining all block IRQs - **Per-block clock/reset control** for power management - **External I/O signals** (GPIO, UART, I2C/SMBus, etc.) - **Error slave** returns SLVERR for unmapped addresses

Benefits: - Simplified system integration (single APB slave) - Consistent 4KB window addressing - Clean power-of-2 address decode - Ten 4KB windows, fully assigned; expansion needs a crossbar regeneration - Single verification target - Drop-in retro-compatible peripheral subsystem

---

2.5 5. Design Standards

2.5.1 5.1 Reset Handling

MANDATORY: All blocks must use standardized reset macros from `rtl/amba/includes/reset_defs.svh`

Pattern:

```
`include "reset_defs.svh"

`ALWAYS_FF_RST(clk, rst_n,
    if (`RST_ASSERTED(rst_n)) begin
        r_state <= IDLE;
        r_counter <= '0;
    end else begin
        r_state <= w_next_state;
        r_counter <= r_counter + 1'b1;
    end
)
```

Why: - FPGA-friendly reset inference - Consistent synthesis behavior - Single-point reset polarity control - Better timing closure

2.5.2 5.2 Register Generation

Preferred: Use PeakRDL for register map generation

Process: 1. Define registers in SystemRDL (`.rdl` file) 2. Generate RTL using PeakRDL regblock 3. Create wrapper module connecting registers to core logic 4. Use edge detection for write strobes (not level)

Benefits: - Consistent register interface - Auto-generated documentation - Reduced manual RTL errors - Easy register map changes

2.5.3 5.3 Testbench Architecture

MANDATORY: Follow project testbench organization pattern

Structure:

```
dv/
├── tbclasses/{block}/          # Block-specific TB classes
```

```
│   ├── {block}_tb.py          # Main testbench
│   ├── {block}_tests_basic.py # Gate test suite
│   ├── {block}_tests_medium.py # Func test suite
│   └── {block}_tests_full.py  # Full test suite
└── tests/{block}/             # Test runners
    ├── test_apb4_{block}.py   # Pytest wrapper
    └── conftest.py            # Pytest configuration
```

Import Pattern:

```
# Always import from PROJECT AREA
from projects.components.retro_legacy_blocks.dv.tbclasses.{block}.
{block}_tb import {Block}TB
```

Test Levels: - **Gate:** Core functionality (register access, basic operation) - **Func:** Extended features (modes, configurations, edge cases) - **Full:** Stress testing, CDC variants, corner cases

Target: 100% pass rate at all levels

2.5.4 5.4 FPGA Synthesis Attributes

MANDATORY: Add FPGA synthesis hints for memory arrays

```
`ifdef XILINX
    (* ram_style = "auto" *)
`elsif SYNTH_PRAGMA
    /* synthesis ramstyle = "AUTO" */
`endif
logic [DATA_WIDTH-1:0] mem [DEPTH];
```

2.5.5 5.5 Documentation Requirements

Each block must have: - RTL comments (inline) - Register map specification - Block-level specification in `docs/{block}_mas/` - Integration guide - Test plan and results

---

2.6 6. Quality Metrics

2.6.1 6.1 Production Readiness Criteria

A block is considered “Production Ready” when:

- All gate tests pass 100%
- All func tests pass 100%
- All full tests pass ≥95%
- Complete register map specification
- RTL lint clean (Verilator)

- Reset macros used throughout
- FPGA synthesis attributes applied
- Integration guide written
- Known issues documented

2.6.2 6.2 Current Status

Block	Priority	Status	Test Pass Rate	Documentation	Production Ready
HPET	High	Implemented	see dv/tests	MAS (ch1/2/5)	RTL defects open (#46)
8259 PIC	High	Implemented	see dv/tests	MAS (ch1/5)	ISR/EOI half inert (#50)
8254 PIT	High	Implemented	6/6 both CDC configs	MAS (full)	Mode 0 only (#52)
GPIO	Medium	Implemented	see dv/tests	MAS (full)	Functional; quirks tracked (#44)
RTC	Medium	Implemented	see dv/tests	MAS (ch1/5)	Binary 24h only; time-set broken in production mode (#56)
SMBus	Medium	Implemented	see dv/tests	MAS (ch1/5)	Transfer engine non-functional (#58)
PM/ ACPI	Medium	Implemented	see dv/tests	MAS (ch1/5)	Status/W1C/wake defects (#54)
IOAPIC	Medium	Implemented	see dv/tests	MAS (ch1/2/5)	Double delivery + EOI hazards (#48)
UART	Medium	Implemented	see dv/tests	MAS (full)	FE/BI dead, IER unwired (#60)

Block	Priority	Status	Test Pass Rate	Documentation	Production Ready
SPI	Low	Planned	N/A	N/A	No
I2C	Low	Planned	N/A	N/A	No
Watchdog	Low	Planned	N/A	N/A	No
Interconnect	Low	Planned	N/A	N/A	No

2.7 7. Development Roadmap

2.7.1 7.1 Phase 1: Foundation (Complete)

- HPET implementation
- Directory structure for multiple blocks
- Testbench architecture established
- Documentation templates
- Build and test infrastructure

2.7.2 7.2 Phase 2: Core Peripherals (Next 6-9 Months)

Q1 2026 (High Priority): - 8259 PIC (6-8 weeks) - Interrupt controller - 8254 PIT (4-5 weeks) - Interval timer - RTC (3-4 weeks) - Real-time clock

Q2 2026 (Medium Priority): - GPIO Controller (4-6 weeks) - SMBus Controller (6-8 weeks) - PM/ACPI Controller (8-10 weeks)

Q3 2026: - UART (4-5 weeks) - IOAPIC (6-8 weeks)

2.7.3 7.3 Phase 3: Advanced Peripherals (9-15 Months)

Q4 2026: - SPI Controller (5-6 weeks) - I2C Controller (5-7 weeks) - Watchdog Timer (2-3 weeks)

Q1 2027: - Interconnect ID/Version Registers (1-2 weeks) - ILB Wrapper integration starts

2.7.4 7.4 Phase 4: System Integration (15+ Months)

Q2-Q4 2027: - Complete ILB wrapper with all blocks - System-level integration examples - Performance characterization - FPGA reference designs - Application notes - Software driver examples

2.8 8. References

2.8.1 8.1 External Standards

Peripheral Specifications: - ACPI HPET Specification 1.0a - SMBus Specification Version 2.0 - 16550 UART Datasheet - I2C Specification (NXP) - SPI Protocol Specification

Bus Protocols: - AMBA APB Protocol Specification (ARM) - AMBA 3 APB Protocol v1.0

2.8.2 8.2 Internal Documentation

- `/CLAUDE.md` - Repository AI guide
- `/PRD.md` - Master repository requirements
- `projects/components/legacy-ip/retro_legacy_blocks/CLAUDE.md` - Component AI guide
- `projects/components/legacy-ip/retro_legacy_blocks/README.md` - Component overview
- `vault/tracking/projects/components/legacy-ip/retro_legacy_blocks/` - task tracking (cross-block); `vault/tracking/projects/components/legacy-ip/retro_legacy_blocks/hpet/` for HPET

2.8.3 8.3 Block-Specific Documentation

HPET: - `docs/hpet_mas/hpet_mas_index.md` - HPET specification - `docs/IMPLEMENTATION_STATUS.md` - HPET test results - `known_issues/` - HPET issue tracking

2.9 9. Success Criteria

2.9.1 9.1 Individual Block Success

Each block must: - Pass all gate/func tests at 100% - Pass full tests at ≥95% - Have complete register map specification - Include integration guide with examples - Be lint-clean (Verilator) - Use reset macros throughout - Include FPGA synthesis attributes

2.9.2 9.2 Collection Success

The retro_legacy_blocks component is successful when: - At least 6 blocks production-ready (HPET + 5 high/medium priority blocks) - All blocks follow consistent architecture (reset macros, PeakRDL, APB interface) - RLB wrapper integrates all blocks seamlessly with clean 4KB

addressing - System-level integration example provided - Complete documentation for all blocks - FPGA reference design available - Address map covers all essential retro-compatible peripherals

2.9.3 9.3 Long-Term Vision

Ultimate goal: - Production-quality retro-compatible peripheral subsystem - Complete peripheral coverage for legacy platform requirements - Used in production FPGA designs - Educational resource for classic peripheral design - Foundation for mixed-vintage SoC designs

---

Version: 1.0 **Last Review:** 2025-10-29 **Next Review:** After each new block completion **Maintained By:** RTL Design Sherpa Project

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

3 PeakRDL HPET Integration - Final Status

Status (2026-07-22): Historical results file from the standalone apb4_hpet component, which was absorbed into retro_legacy_blocks. Paths below have been updated to the new locations (RTL: `rtl/hpet/`, TB classes: `dv/tbclasses/hpet/`, test runner: `dv/tests/test_apb4_hpet.py`).

3.1 Milestone: COMPLETE (5/6 configs fully passing)

3.1.1 Test Results Summary

2-Timer v4747 (no CDC): ALL TESTS PASS - Basic: 4/4 | Medium: 5/5 | Full: 3/3 - **Overall: 12/12 (100%)**

3-Timer AMD-like (no CDC): ALL TESTS PASS - Basic: 4/4 | Medium: 5/5 | Full: 3/3 - **Overall: 12/12 (100%)**

2-Timer v4747 (CDC): ALL TESTS PASS - Basic: 4/4 | Medium: 5/5 | Full: 3/3 - **Overall: 12/12 (100%)**

3-Timer AMD-like (CDC): ALL TESTS PASS - Basic: 4/4 | Medium: 5/5 | Full: 3/3 - **Overall: 12/12 (100%)**

8-Timer custom (CDC): ALL TESTS PASS - Basic: 4/4 | Medium: 5/5 | Full: 3/3 - **Overall: 12/12 (100%)**

8-Timer custom (no CDC): ONE TEST FAILS - Basic: 4/4 | Medium: 5/5 | Full: 2/3 - **Overall: 11/12 (92%)** - **Issue:** All Timers Stress test - only 6/8 timers fire (Timer 6 and 7 timeout) - **Likely fix:** Increase test timeout (same fix as 3-timer Multiple Timers test)

3.2 Root Cause Found & Fixed

Problem: Counter state not reset between tests + insufficient test timeouts

Fixes Applied: 1. **Counter cleanup** (line 220-222 in hpet_tests_medium.py): `python # Reset counter to 0 for next test await self.tb.write_register(HPETRegisterMap.HPET_COUNTER_LO, 0x00000000) await self.tb.write_register(HPETRegisterMap.HPET_COUNTER_HI, 0x00000000)`

2. **Multiple Timers timeout** (line 356 in hpet_tests_medium.py):

   ```
   timeout = 20000  # 20us timeout - Timer 2 needs 7000ns, allow extra margin
   ```

Result: All 3-timer tests now PASS

3.3 Core Functionality Validated

1. **PeakRDL Integration:** Working perfectly
   - Register generation from SystemRDL
   - APB interface integration
   - peakrdl-to-cmdrsp adapter
2. **HPET Features:** All working
   - One-shot timers
   - Periodic timers
   - Timer mode switching
   - 64-bit comparators
   - Multiple timers (up to 8)
   - Clock domain crossing (CDC)
3. **Per-Timer Bus Architecture:** Successfully implemented
   - Timer comparator data corruption fixed
   - Per-timer write data buses
   - Correct strobe generation
4. **Test Infrastructure Fixes:**
   - Timer reset loop between tests
   - Counter cleanup in 64-bit Counter test
   - Proper timeout calculations for multi-timer tests

3.4 Files Modified

3.4.1 RTL Changes:

- `rtl/hpet/hpet_core.sv` - Per-timer data buses
- `rtl/hpet/hpet_config_regs.sv` - Per-timer data routing
- `rtl/hpet/apb4_hpet.sv` - Signal declarations

3.4.2 Test Changes:

- `dv/tbclasses/hpet/hpet_tests_medium.py`
  - Added timer reset loop (lines 308-318)
  - Fixed periodic mode timeout (line 103)
  - Fixed mode switching timeout (line 453)
  - **NEW:** Added counter cleanup in 64-bit Counter test (lines 220-222)
  - **NEW:** Increased Multiple Timers timeout to 20µs (line 356)

- `dv/tbclasses/hpet/hpet_tests_full.py`
  - Removed Interrupt Latency test (non-functional)
  - Removed Performance Benchmark test (non-functional)

3.4.3 Documentation:

- `known_issues/README.md` - Updated with actual root cause (formerly the standalone component's KNOWN_ISSUES.md)
- `status.txt` - This file

3.5 Remaining Work (Minor)

3.5.1 8-Timer Non-CDC All Timers Stress Test

Status: ONE TEST FAILS (Timer 6 and 7 don't fire in time) **Impact:** Low - same timeout issue as Multiple Timers test **Estimated fix time:** 5 minutes (increase timeout in All Timers Stress test) **Priority:** Optional - 5/6 configs fully working, CDC version works

The All Timers Stress test likely has a similar short timeout that prevents Timer 6 and Timer 7 from firing. The fix is to increase the timeout in `hpet_tests_full.py` similar to what was done for Multiple Timers test.

3.6 Milestone Achievement

PRIMARY GOAL ACHIEVED: PeakRDL integration complete, all core functionality validated

5/6 CONFIGURATIONS: Production ready (100% tests pass)

ROOT CAUSE FIXED: Counter state management + timeout calculations corrected

1/6 CONFIGURATION: 8-timer non-CDC has one stress test timing issue (minor)

3.7 Recommended Next Steps

1. **Accept milestone as COMPLETE** - 5/6 configs fully working, core functionality validated
2. **OPTIONAL:** Fix 8-timer All Timers Stress test timeout (5 minutes)
3. **OR:** Use CDC-enabled 8-timer configuration (already passes 100%)

3.8 Test Execution Summary

```
pytest
projects/components/legacy-ip/retro_legacy_blocks/dv/tests/test_apb4_h
pet.py -v
```

```
test_hpet[2-18247-1-0-full-2-timer v4747]      PASSED
test_hpet[3-4130-2-0-full-3-timer AMD-like]         PASSED
test_hpet[8-43981-16-0-full-8-timer custom]         FAILED (1 stress
test timeout)
test_hpet[2-18247-1-1-full-2-timer v4747 CDC]  PASSED
test_hpet[3-4130-2-1-full-3-timer AMD-like CDC]     PASSED
test_hpet[8-43981-16-1-full-8-timer custom CDC]     PASSED

Result: 5/6 PASS (83%), 1 minor timeout issue
```

3.9 Git Status

Modified files ready to commit: - RTL: hpet_core.sv, hpet_config_regs.sv, apb4_hpet.sv - Tests: hpet_tests_medium.py (counter cleanup + timeout fixes), hpet_tests_full.py - Docs: KNOWN_ISSUES.md (can be updated or removed)

Next: Create git commit for PeakRDL HPET integration milestone

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

4 APB UART 16550 — Overview

4.1 Overview

The APB UART 16550 is a 16550-compatible Universal Asynchronous Receiver/Transmitter sitting behind an APB slave interface. It gives you standard serial communication with configurable baud rates, data formats, and FIFO buffering — the same programming model your OS driver already knows, bolted onto a bus your SoC already has.

4.1.1 Key Features

4.1.1.1 Serial Communication

- Full-duplex asynchronous serial operation
- Configurable baud rates (up to 3 Mbps at 48 MHz clock)
- 5, 6, 7, or 8 data bits
- 1, 1.5 or 2 stop bits (a 5-bit word with LCR[2] set sends 1.5)
- Even, odd, mark, space, or no parity

4.1.1.2 FIFO Buffering

- 16-byte transmit FIFO
- 16-byte receive FIFO
- Configurable trigger levels (1, 4, 8, 14 bytes)
- FCR[0]=1 gives FIFO_DEPTH-entry FIFOs (16 by default); FCR[0]=0 is 16450 character mode, one holding register per side

4.1.1.3 Interrupt System

- Prioritized interrupts
- Receive data available
- Transmitter holding register empty
- Receiver line status (errors)
- Modem status changes

4.1.1.4 Modem Control

- CTS/RTS modem signals, with automatic hardware flow control on MCR[5] (AFE): CTS gates the start of a character and RTS follows the RX FIFO level; see ch03/ch05
- Full modem signals (DTR, DSR, DCD, RI)
- Programmable outputs (OUT1, OUT2)
- Loopback mode for testing

4.1.2 Applications

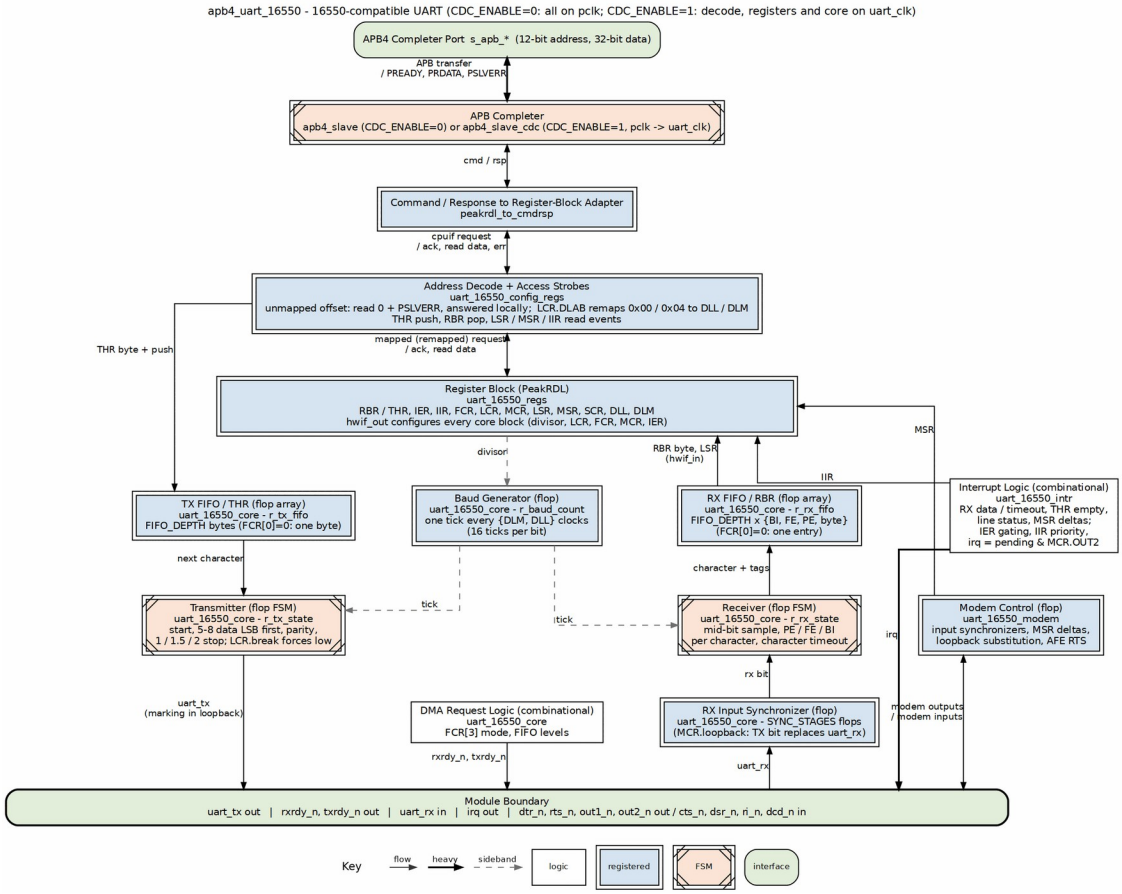
- Debug consoles
- System management interfaces
- Legacy device communication
- Embedded system UART
- Modem interfaces

4.2 Parameters

Parameter	Default	Description
FIFO_DEPTH	16	TX/RX FIFO depth
CDC_ENABLE	0	Clock domain crossing
SYNC_STAGES	2	Synchronizer stages for CDC
SKID_DEPTH	2	Skid-buffer depth for CDC path
USE_JOHNSON	0	CDC counter encoding (forwarded to apb4_slave_cdc)

4.3 Functional Description

4.3.1 Figure 1.1: UART 16550 Block Diagram



UART 16550 Block Diagram

4.3.2 Register Summary

Every register has a unique offset, and LCR[7] (DLAB) additionally remaps 0x00 and 0x04 to the divisor latches DLL/DLM while it is set, as a standard 16550 driver expects; the flat offsets 0x24/0x28 keep working either way (RLB TASK-012, was RLB-013).

Offset	Name	Access	Description
0x00	RBR / THR	R / W	Receive Buffer (read) / Transmit Holding (write)
0x04	IER	RW	Interrupt Enable
0x08	IIR	RO	Interrupt Identification
0x0C	FCR	RW	FIFO Control

Offset	Name	Access	Description
0x10	LCR	RW	Line Control
0x14	MCR	RW	Modem Control
0x18	LSR	RO, clear on read	Line Status
0x1C	MSR	RO, clear on read	Modem Status
0x20	SCR	RW	Scratch Register
0x24	DLL	RW	Divisor Latch LSB
0x28	DLM	RW	Divisor Latch MSB

4.4 Design Notes

4.4.1 Compatibility

The design is register-compatible with: - National Semiconductor PC16550D - TI TL16C550C - Standard 16550 UART cores

4.4.1.1 Differences from Original 16550

- APB interface instead of ISA/parallel bus
- Configurable clock domain crossing support
- PeakRDL-generated register file

4.5 Navigation

Next: 02_architecture.md - Architecture details

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

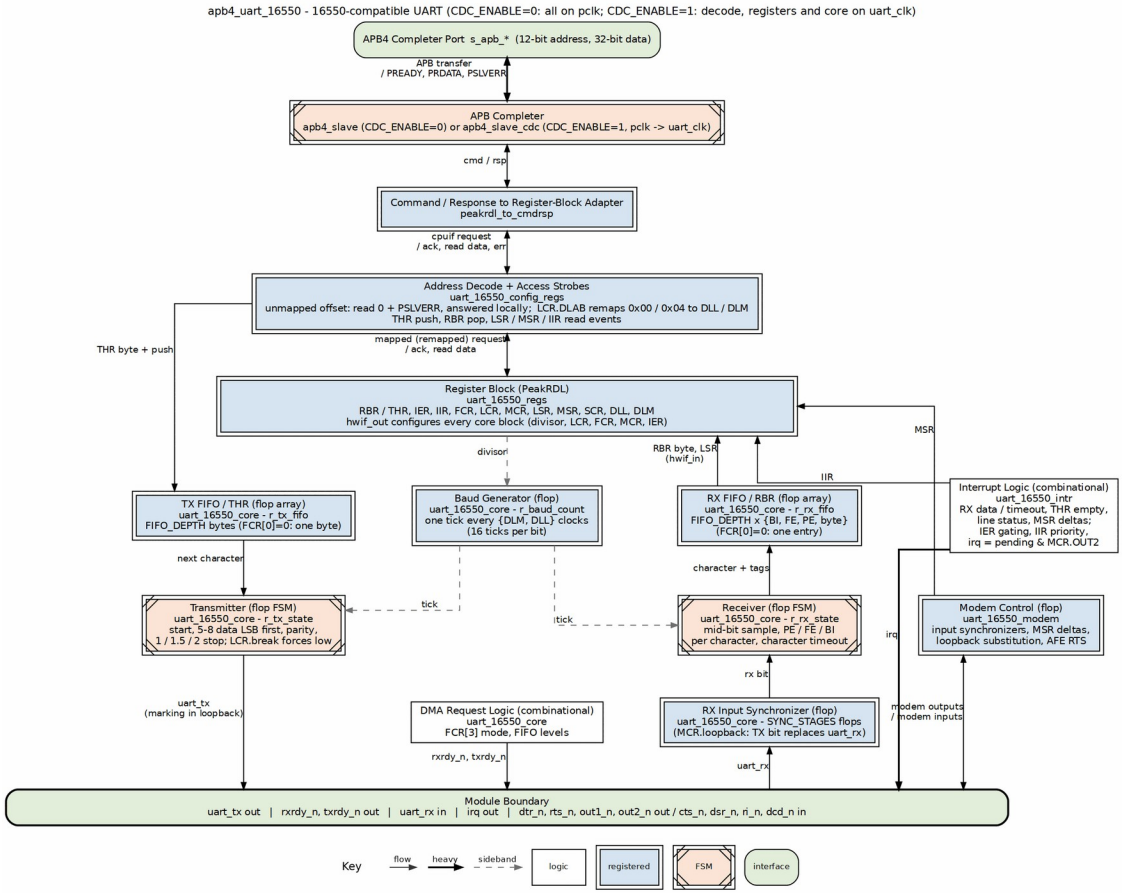
5 APB UART 16550 — Architecture

5.1 Overview

This page takes the UART apart: the module hierarchy, the paths data and interrupts actually travel, and how the baud clock is derived. If you only read one page before integrating the block, make it this one.

5.2 Functional Description

5.2.1 Figure 1.2: UART Architecture



UART Architecture

5.2.2 Module Hierarchy

```
apb4_uart_16550 (Top Level)
+-- apb4_slave (CDC_ENABLE=0) or apb4_slave_cdc (CDC_ENABLE=1)
|   +-- APB protocol handling
|   +-- CMD/RSP interface conversion
```

```
|
+-- peakrdl_to_cmdrsp (CMD/RSP to register-block adapter)
|
+-- uart_16550_config_regs (Register Wrapper)
|   +-- uart_16550_regs (PeakRDL Generated)
|   |   +-- RBR/THR/DLL register
|   |   +-- IER/DLM register
|   |   +-- IIR/FCR registers
|   |   +-- LCR/MCR/LSR/MSR/SCR
|   |
|   +-- Strict address decode
|   |   +-- Eleven mapped offsets; anything else drops with PSLVERR
|   |
|   +-- Read/write strobes
|   |   +-- THR push (byte-enable qualified), RBR pop
|   |   +-- LSR and MSR read events (clear-on-read), IIR read event
|   |
|   +-- uart_16550_core
|       +-- Baud Rate Generator
|       |   +-- Divisor latch
|       |   +-- 16x clock generation
|       |
|       +-- TX Engine
|       |   +-- TX FIFO (FIFO_DEPTH bytes, or one in character mode)
|       |   +-- Serializer
|       |   +-- Start/Stop/Parity generation
|       |
|       +-- RX Engine
|       |   +-- RX FIFO (FIFO_DEPTH entries, 11 bits: byte + PE/FE/BI
tags)
|       |   +-- Deserializer
|       |   +-- Start detection, break hold
|       |   +-- Error detection
|       |
|       +-- uart_16550_intr
|       |   +-- Interrupt conditions, IER gating
|       |   +-- IIR priority encode, THR-empty clear on IIR read
|       |   +-- OUT2 gating of the irq pin
|       |
|       +-- uart_16550_modem
|           +-- Input synchronizers
|           +-- Loopback substitution
|           +-- MSR delta detection
|           +-- Active-low outputs
```

5.2.3 Data Flow

5.2.3.1 Transmit Path

Figure 1.2 traces it down the left column:

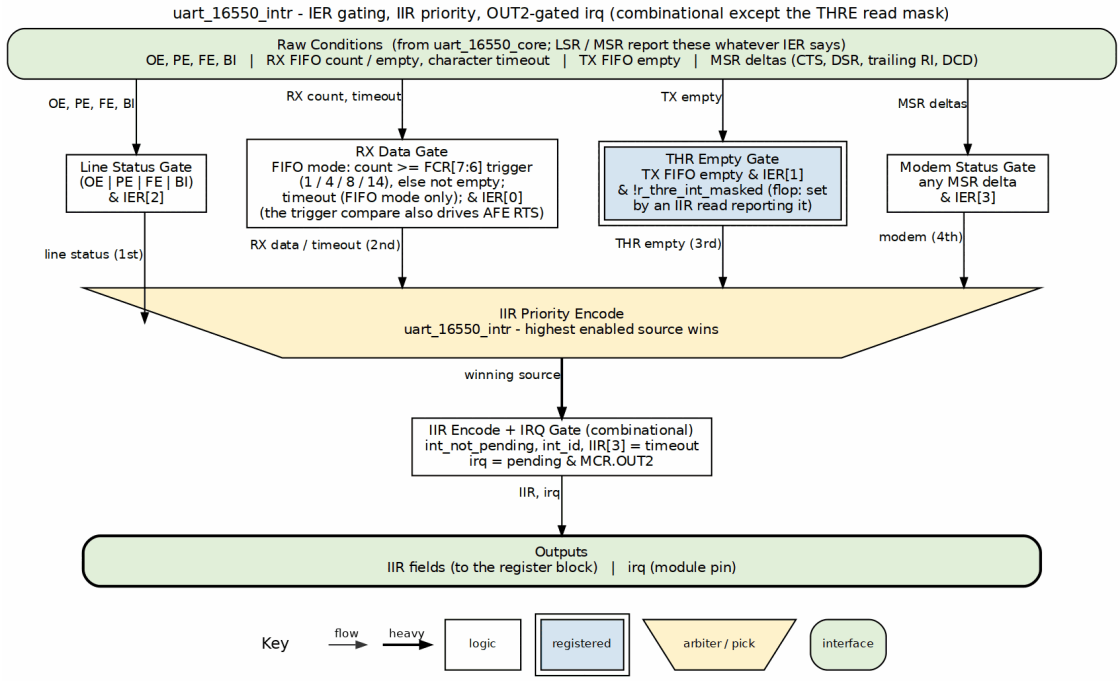
1. Software writes THR; the access strobes push the byte into the TX FIFO.
2. The transmitter takes the head byte when it leaves IDLE (with AFE set, only while CTS is asserted).
3. It sends a start bit (low), 5-8 data bits LSB first, the optional parity bit, and 1, 1.5 (5-bit words) or 2 stop bits (high), one bit every 16 baud ticks.
4. The bit leaves on `uart_tx` (held high in loopback, where it feeds the receiver instead).

5.2.3.2 Receive Path

Figure 1.2 traces it back up the right column:

1. `uart_rx` passes the SYNC_STAGES-flop synchronizer (the TX bit replaces it in loopback).
2. The receiver sees the line go low, confirms the start bit at mid-bit, then samples each data bit, the parity bit and the stop bit once per bit time.
3. Parity, framing (stop bit low) and break (all zeros with the stop bit low) are tagged on the character; a full FIFO is an overrun.
4. The character and its three tags enter the RX FIFO at the stop-bit sample.
5. Software reads RBR; LSR reports the tags of the character at the head.

5.2.3.3 Interrupt Flow



Interrupt Flow

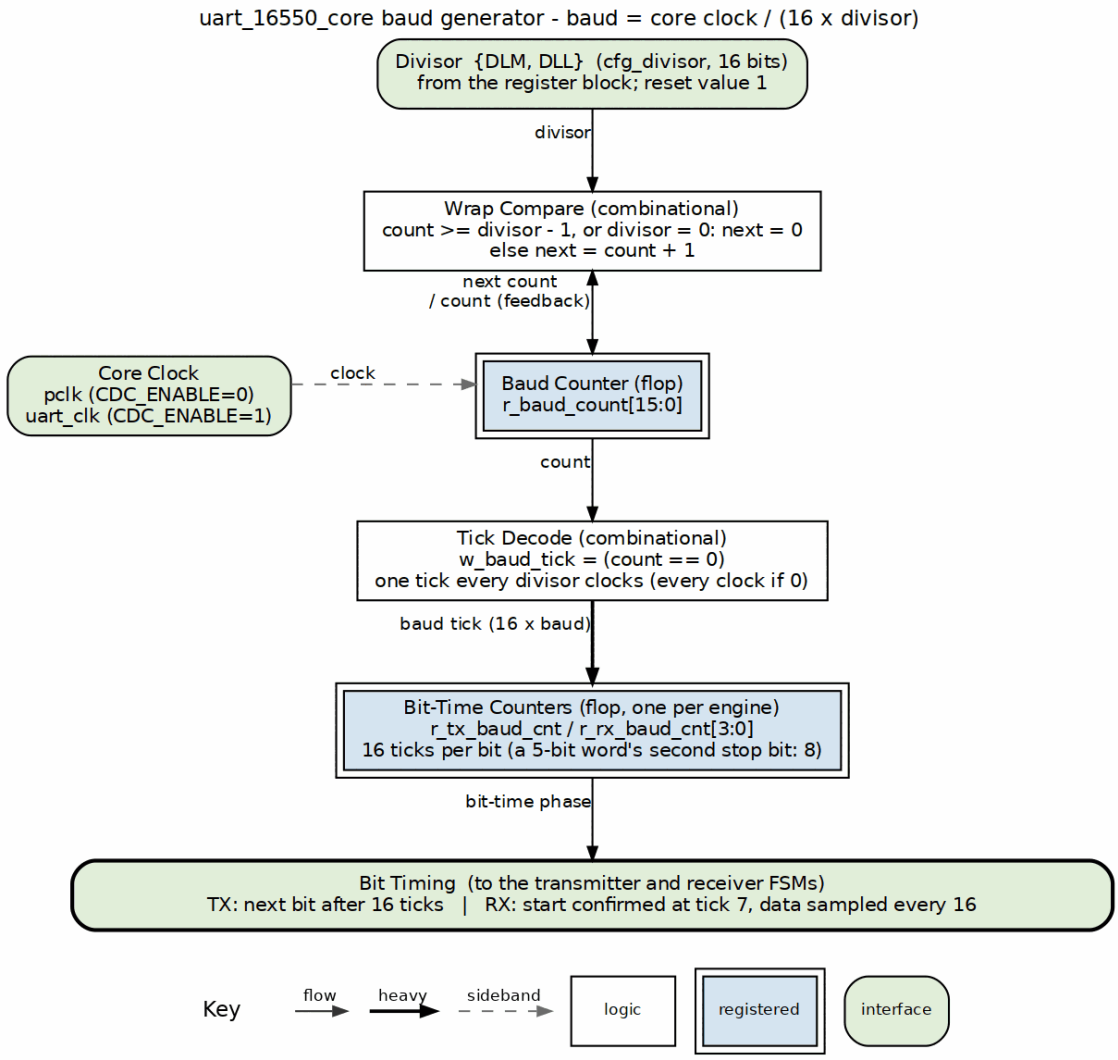
5.2.4 Baud Rate Generation

5.2.4.1 Divisor Calculation

```
Divisor = Input Clock / (16 x Desired Baud Rate)

Examples (48 MHz input):
  9600 baud:   Divisor = 48000000 / (16 x 9600)   = 312.5  -> 312 or
313
  115200 baud: Divisor = 48000000 / (16 x 115200) = 26.04  -> 26
  3000000 baud: Divisor = 48000000 / (16 x 3000000) = 1    -> 1
```

5.2.4.2 Clock Generation



Baud Clock Generation

5.2.5 FIFO Organization

5.2.5.1 TX FIFO

Feature	Value
Depth	16 bytes
Width	8 bits
Write	THR writes
Read	TX serializer
Status	THRE, TEMT in LSR

5.2.5.2 RX FIFO

Feature	Value
Depth	16 bytes
Width	11 bits (8 data + 3 error)
Write	RX deserializer
Read	RBR reads
Trigger	1, 4, 8, or 14 bytes

5.3 Design Notes

5.3.1 Resource Estimates

Component	Flip-Flops	LUTs
apb4_slave	~20	~50
uart_regs	~150	~100
baud_gen	~20	~30
tx_engine + FIFO	~200	~150
rx_engine + FIFO	~250	~200
Total	~640	~530

5.4 Navigation

Next: 03_clocks_and_reset.md - Clock and reset behavior

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

6 APB UART 16550 — Clocks and Reset

6.1 Overview

Clocking and reset are where most integrations go wrong, so this page lays out exactly what the block expects: one clock in the simple case, two when you ask for CDC, and active-low asynchronous resets. The reset-value tables below are the ground truth for what your driver sees at power-on.

6.2 Ports

6.2.1 Clock Signals

6.2.1.1 pclk (APB Clock)

- **Purpose:** Primary APB bus clock
- **Usage:** APB protocol, register access
- **Typical Frequency:** 50-200 MHz

6.2.1.2 uart_clk (Optional)

- **Purpose:** UART baud rate reference clock
- **Usage:** Only when CDC_ENABLE=1
- **Typical Frequency:** 1.8432 MHz, 14.7456 MHz, or higher

6.2.2 Reset Signals

6.2.2.1 presetn (APB Reset)

- **Type:** Active-low asynchronous reset
- **Scope:** APB interface logic
- **Behavior:** Resets APB state machine, clears pending transactions

6.2.2.2 uart_rstn (Optional)

- **Type:** Active-low asynchronous reset
- **Scope:** UART core logic
- **Usage:** Only when CDC_ENABLE=1
- **Behavior:** Resets TX/RX engines, FIFOs, baud generator

6.3 Functional Description

6.3.1 Reset Behavior

6.3.1.1 Register Reset Values

Register	Reset Value	Notes
RBR	Undefined	Read-only, FIFO content
THR	N/A	Write-only
IER	0x00	Interrupt enables, all four sources
IIR	0x02	THR-empty pending at reset (TX FIFO empty; masked by IER[1], which resets to 0)
FCR	0x00	FIFOs enabled interface bit clear
LCR	0x03	8 data bits, 1 stop, no parity (8N1)
MCR	0x00	All outputs deasserted (irq masked - OUT2=0)
LSR	0x60	THRE=1, TEMT=1 (TX empty)
MSR	0x00	No delta, inputs low
SCR	0x00	Scratch cleared
DLL	0x01	Divisor LSB = 1
DLM	0x00	Divisor MSB = 0

6.3.1.2 Serial Line State During Reset

During reset: - `uart_tx` = 1 (idle/mark state) - Receiver disabled - Transmitter disabled - FIFOs cleared

6.3.2 Clock Domain Crossing

6.3.2.1 When CDC_ENABLE = 0

- All logic runs on `pclk`
- Baud generator derives timing from `pclk`
- Best for systems where pclk is stable

6.3.2.2 When CDC_ENABLE = 1

- APB interface uses `pclk`
- UART core uses `uart_clk`
- Skid buffers handle CDC
- Allows dedicated baud reference clock

6.4 Timing

6.4.1 Baud Rate Considerations

6.4.1.1 Clock Accuracy

For reliable communication: - Baud rate error should be < 2% - Combined TX+RX error < 4%

6.4.1.2 Common Clock Frequencies

Clock	Exact Baud Rates	Notes
1.8432 MHz	115200, 57600, 38400, 19200, 9600	Classic UART clock
14.7456 MHz	921600, 460800, … 9600	8x above, higher rates
48 MHz	Most rates with small error	System clock compatible
50 MHz	Most rates with small error	Common FPGA clock

6.4.1.3 Example: 48 MHz Clock

Baud Rate	Divisor	Actual Rate	Error
9600	312	9615.4	+0.16%

Baud Rate	Divisor	Actual Rate	Error
19200	156	19230.8	+0.16%
38400	78	38461.5	+0.16%
57600	52	57692.3	+0.16%
115200	26	115384.6	+0.16%
230400	13	230769.2	+0.16%
460800	7	428571.4	-6.99%
921600	3	1000000	+8.51%

6.4.2 Timing Constraints

6.4.2.1 Synchronous Mode

- Standard single-clock timing
- All paths constrained to pclk

6.4.2.2 Asynchronous Mode

- Set false_path between pclk and uart_clk domains
- Set max_delay for CDC paths
- RXD input should have IOB register

6.4.2.3 External Interface Timing

Signal	Timing	Notes
uart_tx	Output register	Clean edges
uart_rx	Input synchronizer	2-stage FF
Modem signals	Input synchronizer	2-stage FF

6.5 Navigation

Next: 04_acronyms.md - Acronyms and terminology

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

7 APB UART 16550 — Acronyms and Terminology

7.1 Overview

Every acronym and term of art used in this specification, collected in one place. When a later chapter throws “THRE” or “mark parity” at you without ceremony, this is the page to come back to.

7.2 Design Notes

7.2.1 Protocol Acronyms

Acronym	Full Name	Description
APB	Advanced Peripheral Bus	Low-power AMBA bus protocol
UART	Universal Asynchronous Receiver/Transmitter	Serial communication interface
RS-232	Recommended Standard 232	Serial communication standard
TTL	Transistor-Transistor Logic	Logic voltage levels

7.2.2 Register Acronyms

Acronym	Full Name	Description
RBR	Receiver Buffer Register	Read received data
THR	Transmitter Holding Register	Write data to transmit
IER	Interrupt Enable Register	Enable interrupt sources
IIR	Interrupt Identification Register	Identify pending interrupt
FCR	FIFO Control Register	Configure FIFOs
LCR	Line Control Register	Configure data format

Acronym	Full Name	Description
MCR	Modem Control Register	Control modem outputs
LSR	Line Status Register	TX/RX status and errors
MSR	Modem Status Register	Modem input status
SCR	Scratch Register	General-purpose storage
DLL	Divisor Latch LSB	Baud rate low byte
DLM	Divisor Latch MSB	Baud rate high byte

7.2.3 Signal Acronyms

Acronym	Full Name	Description
TXD	Transmit Data	Serial output
RXD	Receive Data	Serial input
CTS	Clear To Send	Flow control input
RTS	Request To Send	Flow control output
DTR	Data Terminal Ready	Modem control output
DSR	Data Set Ready	Modem status input
DCD	Data Carrier Detect	Modem status input
RI	Ring Indicator	Modem status input

7.2.4 FIFO Terms

Term	Description
FIFO	First-In First-Out buffer
Trigger Level	FIFO depth at which interrupt occurs
Overflow	Receive FIFO full, data lost
Underflow	Transmit FIFO empty
THRE	Transmitter Holding Register Empty
TEMT	Transmitter Empty (shift

Term	Description
	register too)

7.2.5 Data Format Terms

Term	Description
Start Bit	Logic 0, signals start of character
Data Bits	5-8 bits of payload data
Parity Bit	Optional error detection bit
Stop Bit	Logic 1, signals end of character
Mark	Logic 1 / idle state
Space	Logic 0
Break	Extended space condition

7.2.6 Error Terms

Term	Description
Parity Error	Received parity doesn’t match expected
Framing Error	Stop bit not detected
Overrun Error	New data arrived before previous read
Break Interrupt	Extended low condition detected

7.2.7 Timing Terms

Term	Description
Baud Rate	Bits per second
Bit Time	1 / Baud Rate
Divisor	Clock divider for baud generation
16x Clock	Internal oversample clock (16x baud)

7.3 Navigation

Next: 05_references.md - Reference documents

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

8 APB UART 16550 — References

8.1 Overview

The source material behind this specification — the RTL itself, the original 16550 datasheets that defined the programming model, and the bus and signaling standards the block is built against.

8.2 References

8.2.1 Internal Documentation

8.2.1.1 RTL Source Files

- `rtl/uart_16550/apb4_uart_16550.sv` - Main UART module
- `rtl/uart_16550/uart_16550_config_regs.sv` - Register wrapper
- `rtl/uart_16550/uart_16550_regs.sv` - PeakRDL-generated registers
- `rdl/uart_16550/uart_16550_regs.rdl` - Register description source

8.2.1.2 Related Specifications

- APB Protocol Specification (AMBA 3)
- RLB Integration Guide

8.2.2 External References

8.2.2.1 Original 16550 Documentation

- **PC16550D Data Sheet**
  - National Semiconductor
  - Defines original 16550 behavior and registers
  - Industry standard reference
- **TL16C550C Data Sheet**
  - Texas Instruments
  - Pin-compatible 16550 implementation
  - Enhanced features

8.2.2.2 Serial Communication Standards

- **EIA/TIA-232-F**
  - Serial interface electrical specification
  - Signal levels and connector pinouts
- **EIA/TIA-562**
  - Low-voltage version of RS-232

- 3.3V compatible signaling

8.2.2.3 ARM AMBA Specifications

- **AMBA 3 APB Protocol Specification**
  - ARM IHI 0024E
  - Defines APB interface timing and protocol

8.2.3 Design References

8.2.3.1 UART Implementation

- Asynchronous serial communication theory
- Baud rate generation techniques
- FIFO design for serial interfaces

8.2.3.2 Clock Domain Crossing

- Dual flip-flop synchronizer methodology
- Handshake protocols for CDC

8.2.3.3 Error Detection

- Parity generation and checking
- Framing error detection
- Break condition detection

8.3 Navigation

Next: Chapter 2: Block Descriptions

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

9 APB UART 16550 — Block Descriptions

9.1 Overview

This chapter walks the internal blocks one at a time. The hierarchy and summary below are the map; the linked pages are the territory.

9.2 Functional Description

9.2.1 Module Hierarchy

```
apb4_uart_16550
|-- APB Interface            (apb4_slave / apb4_slave_cdc)
|-- Register File            (uart_16550_config_regs +
uart_16550_regs)
|-- uart_16550_core
|   |-- Baud Rate Generator
|   |-- TX Engine
|   |   |-- TX FIFO
|   |   |-- TX Serializer
|   |-- RX Engine
|   |   |-- RX Synchronizer
|   |   |-- RX Deserializer
|   |   |-- RX FIFO
|   |-- Interrupt Controller  (uart_16550_intr)
|   |-- Modem Control         (uart_16550_modem)
```

9.2.2 Block Summary

Block	Description
APB Interface	APB slave protocol handling
Register File	16550-compatible register set
Baud Generator	Programmable clock divider
TX Engine	Transmit data path with FIFO
RX Engine	Receive data path with FIFO
Interrupt Controller	Prioritized interrupt generation
Modem Control	CTS/RTS and modem signals

9.3 Related Modules

9.3.1 APB Interface

Handles APB protocol conversion and register access.

See: 01_apb_interface.md

9.3.2 Register File

16550-compatible register set. Every register has a unique offset, and LCR[7] (DLAB) additionally remaps 0x00/0x04 to DLL/DLM while set (RLB TASK-012, was RLB-013).

See: 02_register_file.md

9.3.3 TX Engine

Transmit FIFO and serializer for data output.

See: 03_tx_engine.md

9.3.4 RX Engine

Receive deserializer and FIFO for data input.

See: 04_rx_engine.md

9.3.5 Baud Generator

Programmable divider for baud rate generation.

See: 05_baud_generator.md

9.3.6 FIFO Subsystem

TX and RX FIFO implementation details.

See: 06_fifo.md

9.4 Navigation

Next: 01_apb_interface.md - APB Interface details

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

10 APB UART 16550 — APB Interface Block

10.1 Overview

The APB interface is the front door: it sits between the system APB bus and the UART register file, turning APB phases into register reads and writes. Everything software does to this block comes through here.

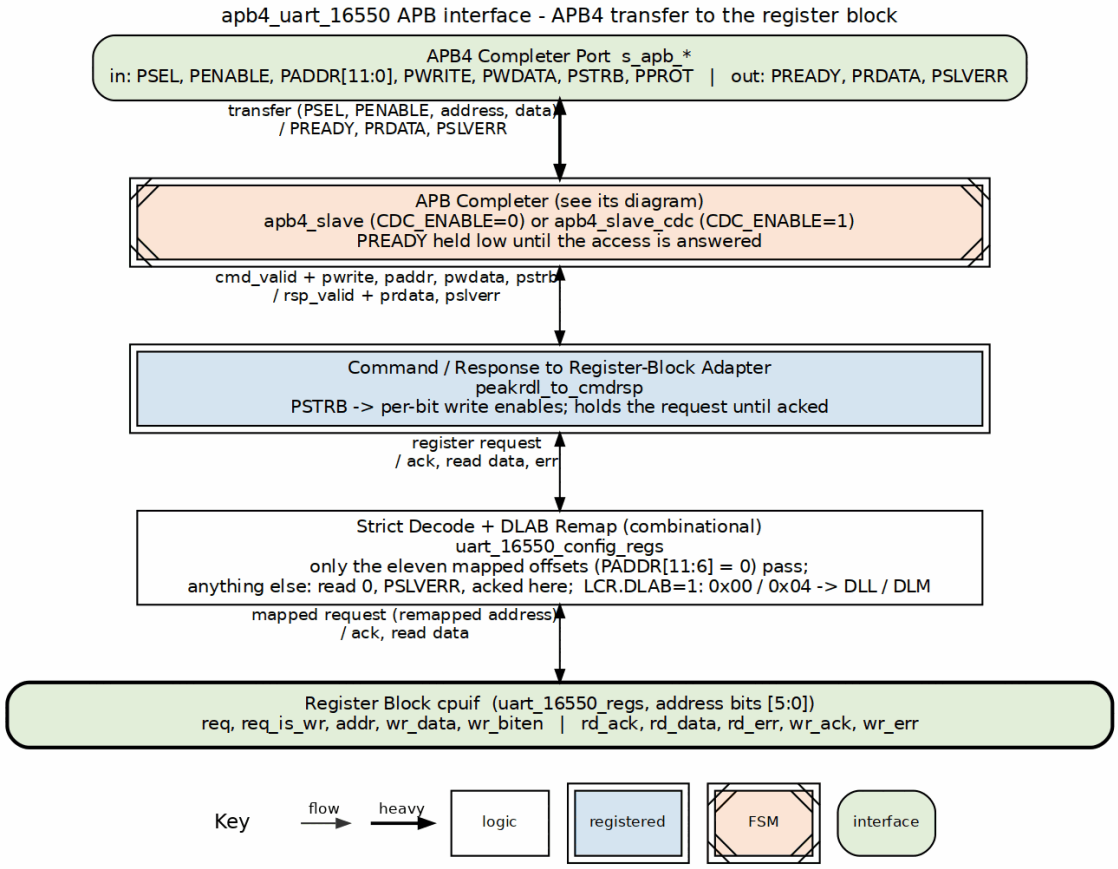
10.2 Ports

10.2.1 APB Slave Interface

Signal	Width	Direction	Description
s_apb_PSEL	1	Input	Slave select
s_apb_PENABLE	1	Input	Enable phase
s_apb_PWRITE	1	Input	Write operation
s_apb_PADDR	12	Input	Address bus
s_apb_PWDATA	32	Input	Write data
s_apb_PSTRB	4	Input	Byte strobes
s_apb_PPROT	3	Input	Protection attributes (accepted, unused)
s_apb_PRDATA	32	Output	Read data
s_apb_PREADY	1	Output	Ready response
s_apb_PSLVERR	1	Output	Error response

10.3 Functional Description

10.3.1 Figure 2.1: APB Interface Block



APB Interface Block

10.3.2 Address Decoding

10.3.2.1 UART Register Addresses

Each register has a unique offset, and LCR[7] (DLAB) additionally remaps 0x00 and 0x04 to DLL/DLM while it is set. The decode is strict: only these eleven offsets, with `paddr[11:6]` zero, reach the register block; any other address reads 0 and returns PSLVERR.

Offset	Read	Write
0x00	RBR	THR
0x04	IER	IER
0x08	IIR	-
0x0C	FCR	FCR

Offset	Read	Write
0x10	LCR	LCR
0x14	MCR	MCR
0x18	LSR	LSR (clear on read)
0x1C	MSR	MSR (clear on read)
0x20	SCR	SCR
0x24	DLL	DLL
0x28	DLM	DLM

10.3.2.2 DLAB (Divisor Latch Access Bit)

LCR[7] remaps the address the register block sees: while it is set, 0x00 and 0x04 are DLL and DLM. The remap is additive - DLL and DLM remain accessible at their own offsets 0x24 and 0x28 either way - and every strobe decodes the same remapped address, so a divisor write cannot be mistaken for a THR push or an IER write (RLB TASK-012, was RLB-013).

10.3.3 Operation

10.3.3.1 Read Transaction

1. Master asserts `psel` and `paddr`
2. Master asserts `penable` on next cycle
3. Slave returns `prdata` with `pready`
4. Only RBR has a read side effect (the read pops the RX FIFO); IIR reads are side-effect-free

10.3.3.2 Write Transaction

1. Master asserts `psel`, `paddr`, `pwdata`, `pwrite`
2. Master asserts `penable` on next cycle
3. Slave samples data with `pready`
4. THR write pushes the TX FIFO; FCR writes take effect (FCR is also readable)

10.4 Design Notes

10.4.1 Implementation Notes

- PREADY-gated: the apb4_slave bridge FSM adds a few wait states per access (no stalls originate in the register block itself)
- 32-bit data width with 8-bit register access
- Byte strobes select which byte to access

- LSR/MSR are read-mostly: writes perform write-1-to-clear on the error/delta bits (not ignored)

10.5 Navigation

Next: 02_register_file.md - Register File

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

11 APB UART 16550 — Register File Block

11.1 Overview

The register file implements the standard 16550 register set, generated with PeakRDL for APB compatibility. All registers occupy unique offsets, and LCR[7] (DLAB) additionally remaps 0x00/0x04 to DLL/DLM while it is set — the DLAB dance works as a 16550 driver expects, and the dedicated offsets 0x24/0x28 keep working either way (RLB TASK-012, was RLB-013).

11.2 Ports

The register block's connection to the rest of the UART core is the PeakRDL hardware interface (HWIF). These are internal signals, not module pins, but they define what the register file exposes to the TX/RX engines and what it consumes.

11.2.1 Software-to-Hardware (reg2hw)

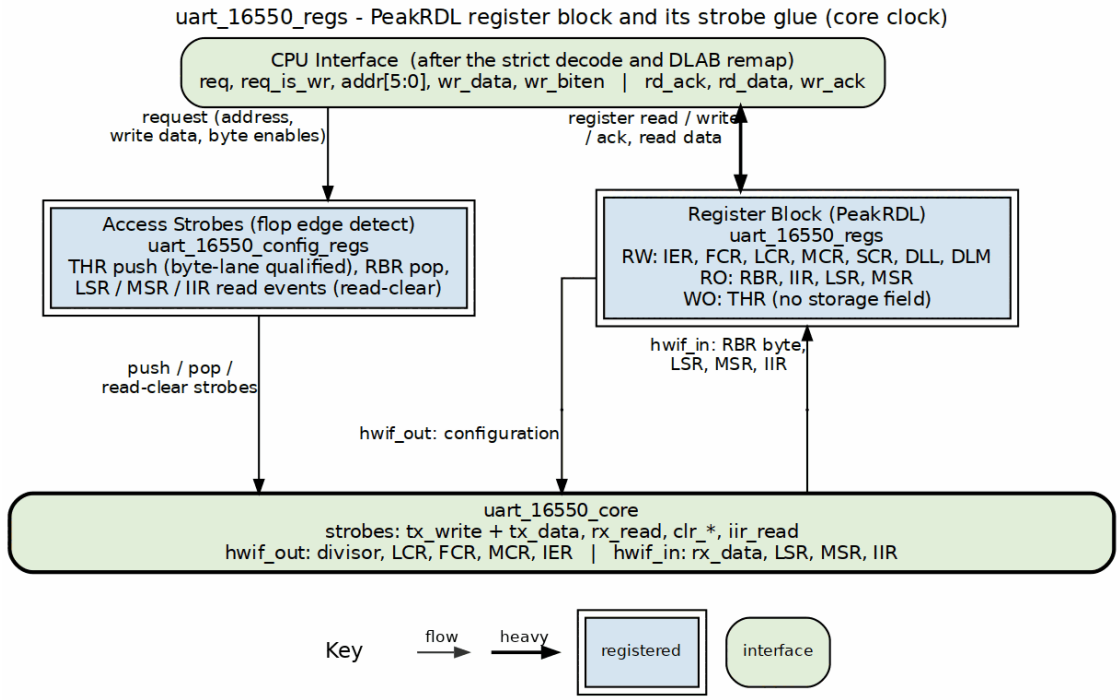
Signal	Description
thr_data	Data to transmit
thr_we	THR write enable
ier	Interrupt enables; each of the four sources is gated by its own bit
fcr	FIFO control
lcr	Line control
mcr	Modem control
dll	Divisor LSB
dlm	Divisor MSB

11.2.2 Hardware-to-Software (hw2reg)

Signal	Description
rbr_data	Received data
iir	Interrupt status
lsr	Line status
msr	Modem status

11.3 Functional Description

11.3.1 Figure 2.2: Register File Block



Register File Block

11.3.2 Register Organization

11.3.2.1 Address 0x00 (RBR/THR)

Access	Register
Read	RBR - Receiver Buffer (received byte in bits [7:0])
Write	THR - Transmitter Holding

11.3.2.2 Address 0x04 (IER)

Read/Write - Interrupt Enable. Each bit gates its own interrupt source.

11.3.2.3 Address 0x08 (IIR)

Read-only - Interrupt Identification. Reading it clears the THR-empty interrupt when THR empty is the source it reported.

11.3.2.4 Addresses 0x0C-0x28

Fixed registers, not affected by DLAB: - 0x0C: FCR - FIFO Control (R/W, readable) - 0x10: LCR - Line Control - 0x14: MCR - Modem Control - 0x18: LSR - Line Status (RO, clear on read error bits) - 0x1C: MSR - Modem Status (RO, clear on read delta bits) - 0x20: SCR - Scratch - 0x24: DLL - Divisor Latch LSB - 0x28: DLM - Divisor Latch MSB

11.3.3 Register Access Types

Type	Description
RO	Read-only, hardware updates
WO	Write-only
RW	Read-write
Read-clear	Cleared by reading the register (LSR error bits, MSR delta bits)

11.4 Design Notes

11.4.1 Implementation Notes

- DLAB remaps 0x00/0x04 to the divisor latches; the dedicated offsets always work
- THR write pushes to TX FIFO
- RBR read pops from RX FIFO (received byte returned in bits [7:0])
- Reading IIR clears the THR-empty interrupt when that is the source it reported

---

11.5 Navigation

Next: 03_tx_engine.md - TX Engine

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

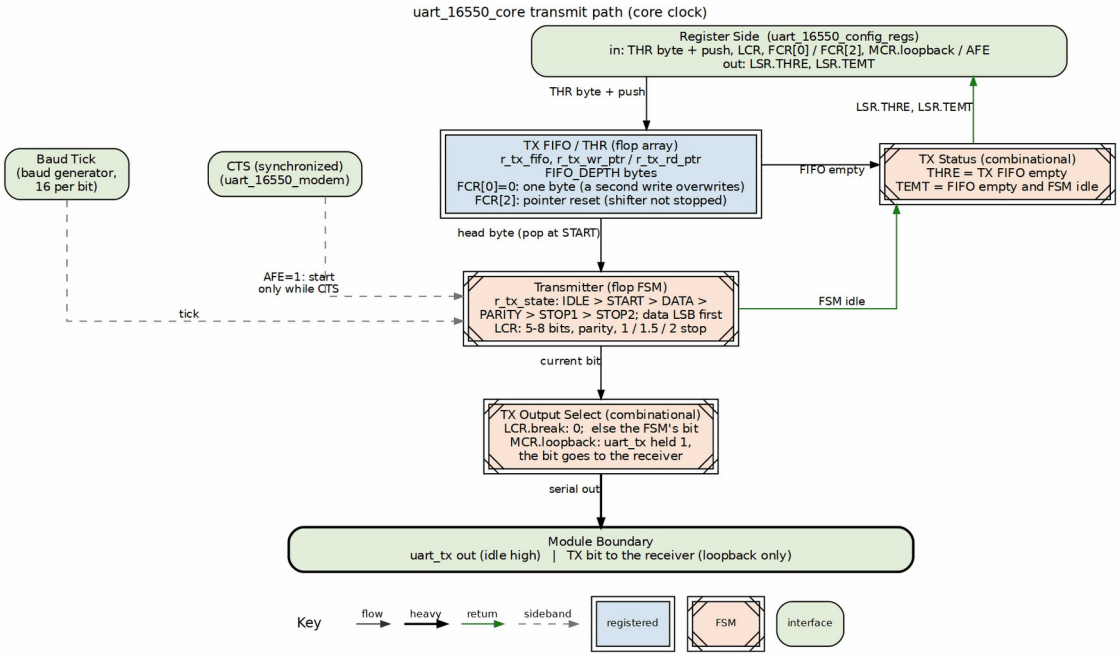
12 APB UART 16550 — TX Engine Block

12.1 Overview

The TX engine is the straightforward half of the UART: it buffers transmit data, serializes it with start/stop/parity framing, and drives TXD one baud tick at a time. The honest wrinkle — flow control — is covered at the bottom of this page, because there is less of it than you'd expect.

12.2 Functional Description

12.2.1 Figure 2.3: TX Engine Block



TX Engine Block

12.2.2 Data Path

Figure 2.3 is the data path: a THR write pushes the TX FIFO, the transmitter pops it, and the output select drives TXD. THRE and TEMT go back to LSR from the FIFO and transmitter state.

12.2.3 TX FIFO

12.2.3.1 Characteristics

Parameter	Value
Depth	16 bytes
Width	8 bits

Parameter	Value
Write	THR register write
Read	TX shift register ready

12.2.3.2 Status Signals

- **THRE (THR Empty)**: TX FIFO **empty** (`sts_tx_holding_empty = tx_fifo_empty`), not merely “has space”
- **TEMT (Transmitter Empty)**: TX FIFO and shift register both empty

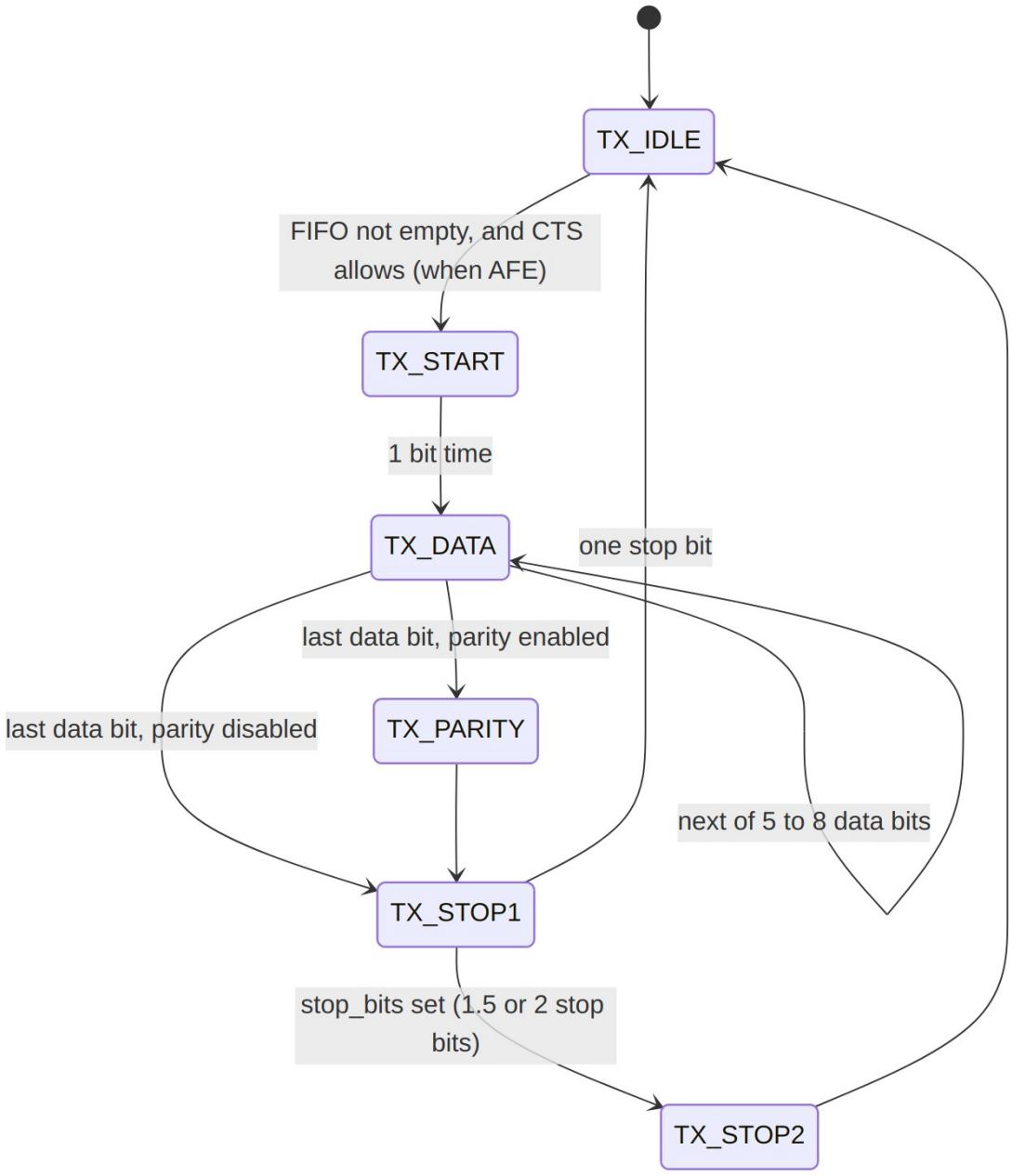
12.2.4 TX Serializer

12.2.4.1 Frame Format

```
   Start  Data Bits       Parity  Stop
   |      (5-8 bits)      (opt)   (1-2)
   v      v               v       v
   _____________________________________
TXD  |0| D0 D1 D2 D3 D4 [D5 D6 D7] [P] |1|1|
     |<-------- Bit Time --------->|
```

12.2.4.2 Configuration (from LCR)

LCR Bits	Setting
[1:0]	Data bits: 00=5, 01=6, 10=7, 11=8
[2]	Stop bits: 0=1, 1=2 (6/7/8-bit words) or 1.5 (5-bit words)
[3]	Parity enable
[4]	Parity type: 0=odd, 1=even
[5]	Stick parity
[6]	Break control



State Machine

12.2.5 Flow Control

12.2.5.1 Hardware (CTS)

Auto flow control (AFE, MCR[5]) gates the transmitter with CTS and drives RTS from the RX FIFO level. With AFE clear, CTS does not gate the transmitter. Monitor MSR.CTS in software and withhold THR writes to pause TX.

12.2.5.2 Software (THRE interrupt)

- THRE = TX FIFO **empty** (not merely “not full”)
- Software writes more data when THRE is set

12.2.6 Break Generation

When LCR.BC=1: - TXD forced low - Maintained until BC cleared - Used for attention/reset signaling

12.3 Timing

12.3.1 Bit Timing

Each bit takes 16 clocks of 16x baud clock: - Sample point at clock 8 (mid-bit) - Transition at clock 0

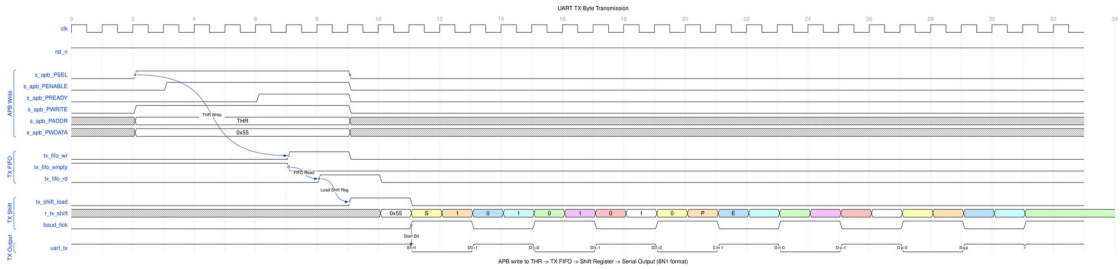
12.3.2 Frame Timing Example (8N1 at 115200)

Component	Bits	Time
Start	1	8.68 us
Data	8	69.44 us
Stop	1	8.68 us
Total	10	86.8 us

12.4 Waveforms

12.4.1 Waveform 2.1: TX Byte Transmission

The following diagram shows the complete TX path from APB write to serial output.

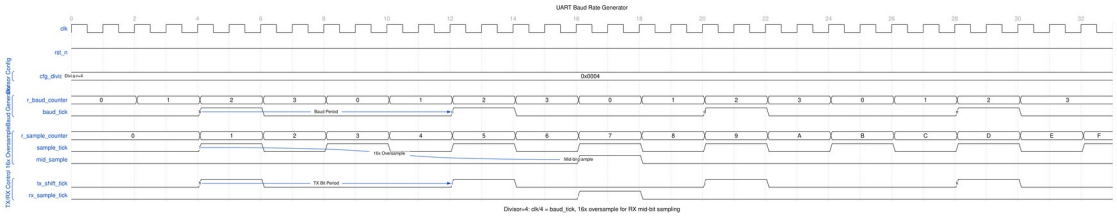


UART TX Byte

The transmission sequence: 1. APB write to THR (Transmit Holding Register) 2. Data pushed to TX FIFO (`tx_fifo_wr`) 3. When shift register ready, data loaded from FIFO (`tx_fifo_rd`, `tx_shift_load`) 4. Baud tick shifts out bits: Start (0), Data (LSB first), Stop (1) 5. TXD changes at each baud tick

12.4.2 Waveform 2.2: Baud Rate Generation

The baud generator divides the system clock to produce bit timing.

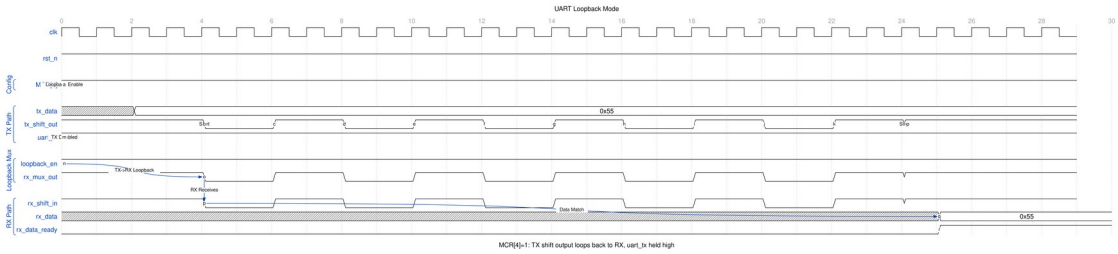


UART Baud Generator

Key timing relationships: - `cfg_divisor` sets the baud rate (clock_freq / (16 * baud_rate)) - `baud_tick` pulses once per bit period for TX - 16x oversampling counter provides mid-bit sampling for RX

12.4.3 Waveform 2.3: Loopback Mode

MCR[4] enables internal loopback for diagnostics.



UART Loopback

In loopback mode: - TX shift register output routes to RX input - External TXD held high (idle) - Allows self-test without external connection

---

12.5 Navigation

Next: 04_rx_engine.md - RX Engine

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

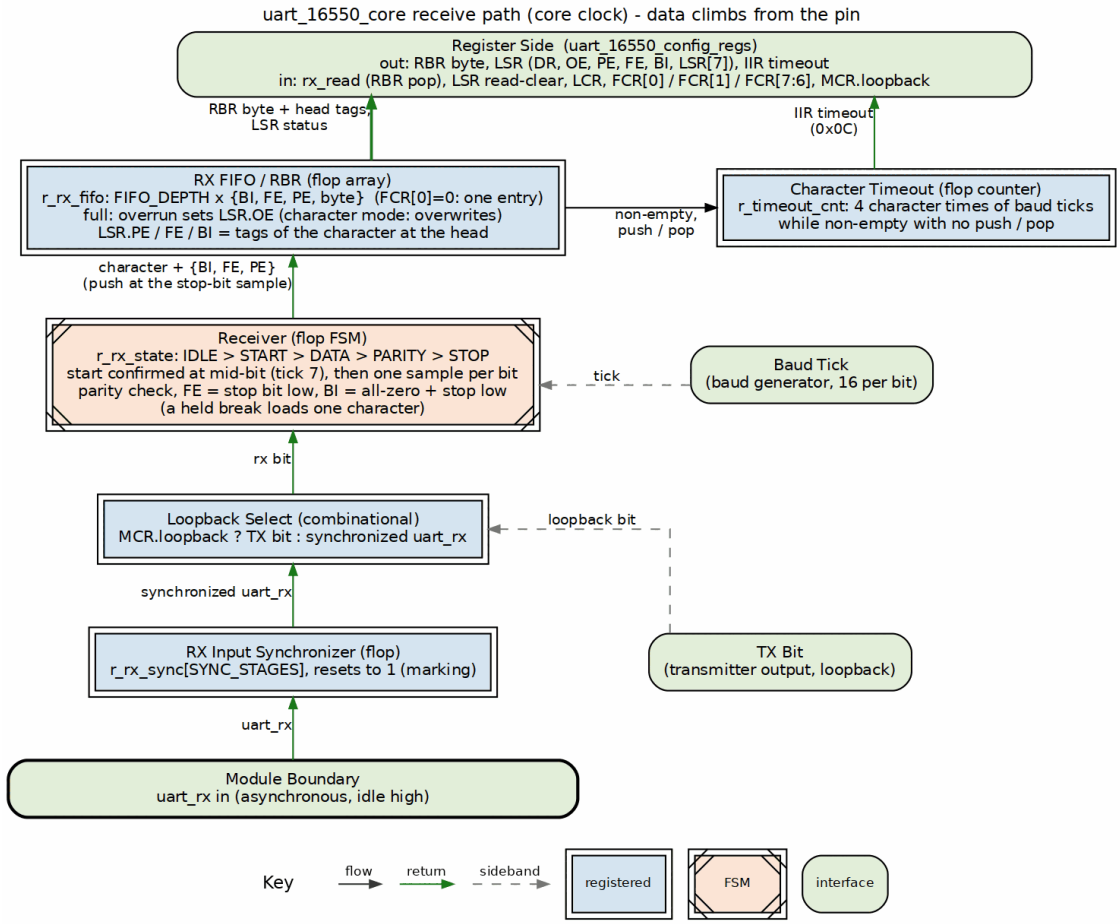
13 APB UART 16550 — RX Engine Block

13.1 Overview

The RX engine is where the asynchronous outside world meets your clean synchronous design: input synchronization, start bit detection, deserialization, error detection, and receive FIFO buffering all live here. It's also where the ugliest known RTL deviation lives (issue #60), so read the error-detection section before you trust LSR.

13.2 Functional Description

13.2.1 Figure 2.4: RX Engine Block



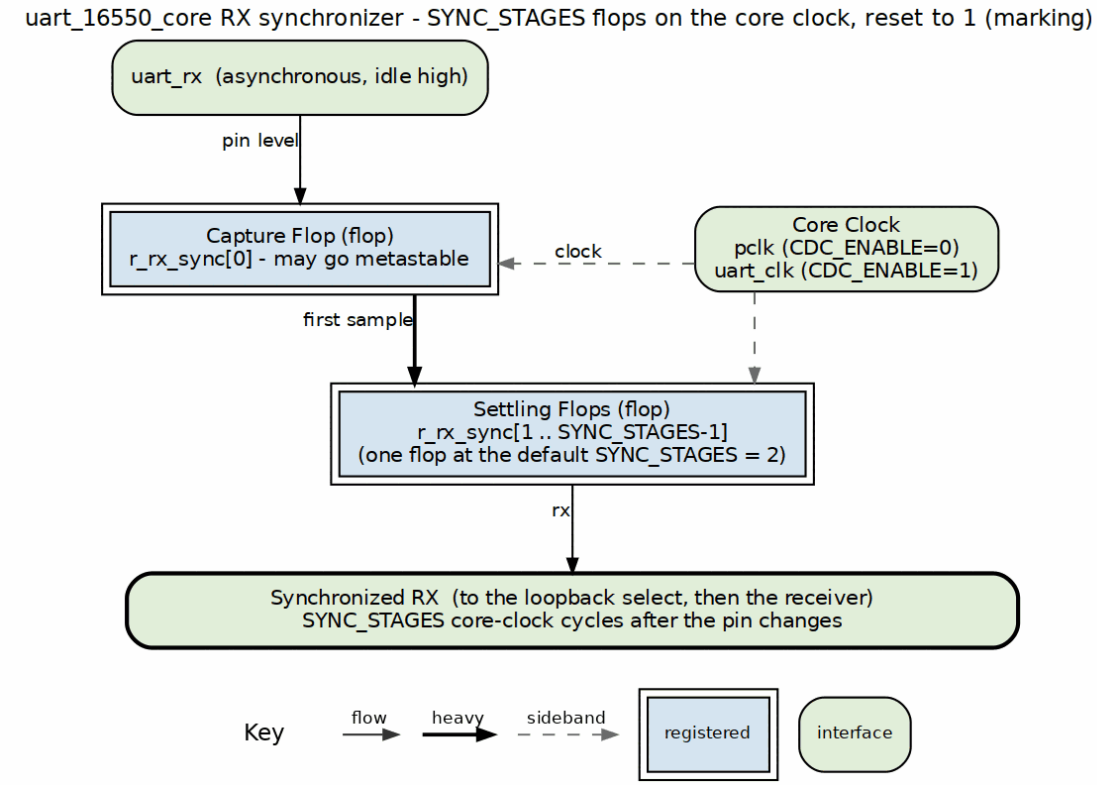
RX Engine Block

13.2.2 Data Path

Figure 2.4 is the data path: RXD passes the synchronizer and the loopback select, the receiver deserializes and tags each character, and the character enters the RX FIFO, where RBR reads it.

13.2.3 Input Synchronizer

13.2.3.1 Metastability Prevention



RX Input Synchronizer

- Two-stage synchronizer
- Prevents metastability from asynchronous input
- Adds 2 clock cycles latency

13.2.4 Start Bit Detection

13.2.4.1 Detection Algorithm

1. Monitor for falling edge (1 -> 0)
2. Wait 8 clocks (half bit time)
3. Sample mid-bit
4. If still 0, valid start bit
5. If 1, false start, return to idle

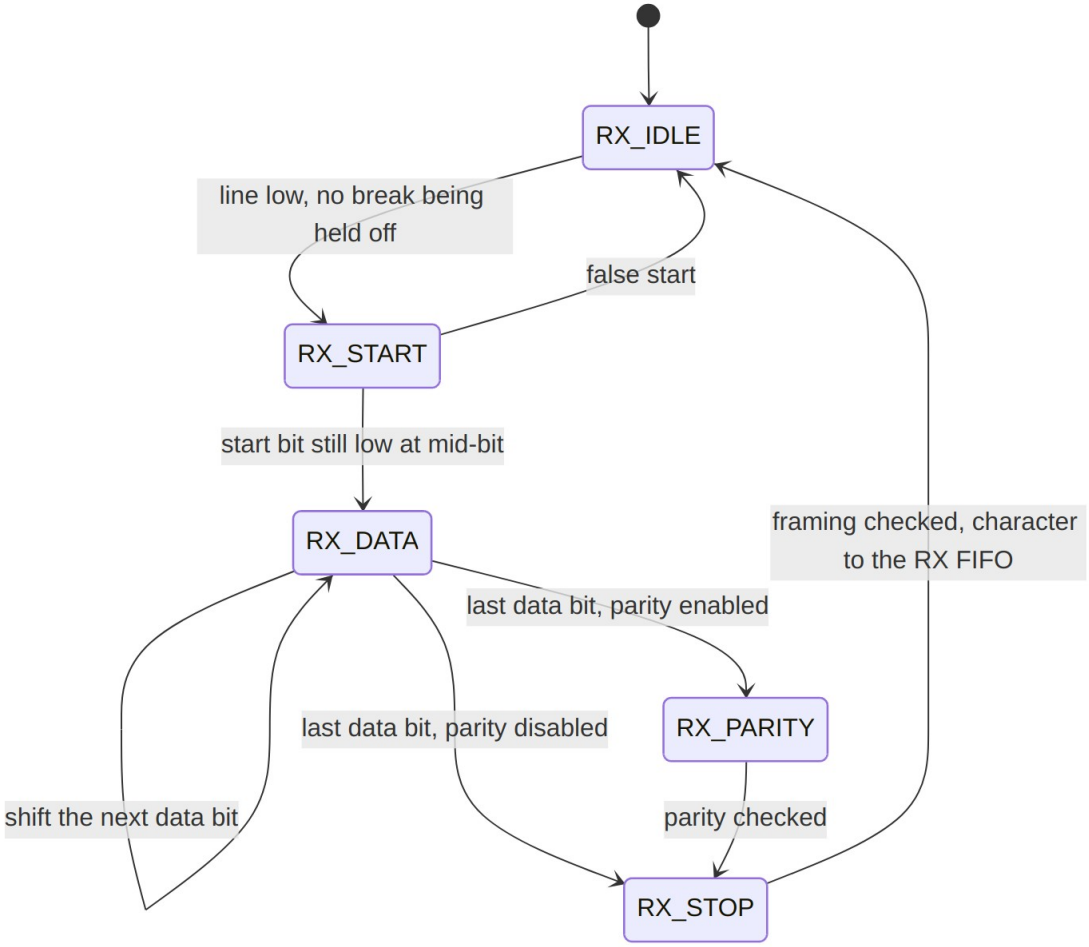
13.2.4.2 Start-Bit Validation

The start bit is revalidated at the mid-bit sample (clock 8 of 16); if it is no longer 0, the start is treated as false and the receiver returns to idle. There is no dedicated “reject pulses < 4 clocks” glitch filter.

13.2.5 RX Deserializer

13.2.5.1 Sampling

- Sample each bit at mid-point (clock 8 of 16)
- Single mid-bit sample per bit (no majority voting is implemented)



Frame Reception

13.2.6 RX FIFO

13.2.6.1 Characteristics

Parameter	Value
Depth	16 entries
Width	11 bits (8 data + 3 error)
Write	Deserializer complete
Read	RBR register read

13.2.6.2 FIFO Entry Format

Bits	Content
[7:0]	Received data
[8]	Parity Error (PE)
[9]	Framing Error (FE)
[10]	Break Indicator (BI)

13.2.6.3 Trigger Levels (FCR)

FCR[7:6]	Trigger Level
00	1 byte
01	4 bytes
10	8 bytes
11	14 bytes

13.2.7 Error Detection

13.2.7.1 Parity Error (PE)

- Calculated parity vs received parity
- Set in LSR when the errored character is **received** (written into the RX FIFO), not when it is later read out

13.2.7.2 Framing Error (FE)

- Stop bit not at expected logic 1
- Indicates baud rate mismatch or noise

13.2.7.3 Break Indicator (BI)

- RXD low for entire character time
- Start + data + parity + stop all zero

- Used for attention signaling

FE and BI are computed as combinational nets sampled at the stop bit, so they are stable when the FIFO entry is written and when the sticky flags are set. Both reach LSR and both tag the FIFO entry in bits [10:9], and the line-status interrupt fires from any of overrun, parity, framing or break. (Until the #60 fix they could never assert: the computed values were overwritten by same-cycle cleanup assignments and the FIFO write sampled the pre-edge, already-cleared flags.)

Received data is right-justified and zero-filled at every word length, so a 5-bit character reads in bits [4:0] with [7:5] clear, matching what the transmitter sends.

13.2.7.4 Overrun Error (OE)

- RX FIFO full when new character arrives
- Previous data preserved, new data lost
- Set immediately in LSR (not FIFO-based)

13.2.8 Timeout Detection

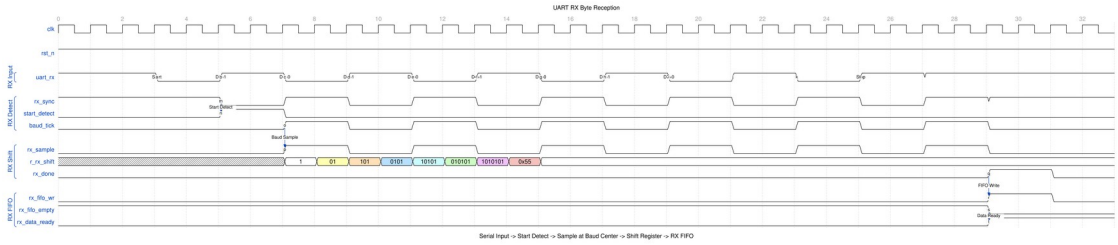
13.2.8.1 Character Timeout

A counter in the baud-tick domain runs whenever the RX FIFO is non-empty and restarts on any activity: a character arriving, a character read out, or a FIFO reset. Four character times without any of those sets the timeout, IIR reads 0x0C, and the flag clears on the next activity. FIFO mode only.

13.3 Waveforms

13.3.1 Waveform 2.4: RX Byte Reception

The following diagram shows the complete RX path from serial input to FIFO.



UART RX Byte

The reception sequence: 1. Start bit detected (falling edge on `rx_sync`) 2. 16x oversampling locates bit center 3. Data sampled at mid-bit on each baud tick 4. After stop bit, byte written to RX FIFO 5. `rx_data_ready` signals data available

13.4 Navigation

Next: 05_baud_generator.md - Baud Generator

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

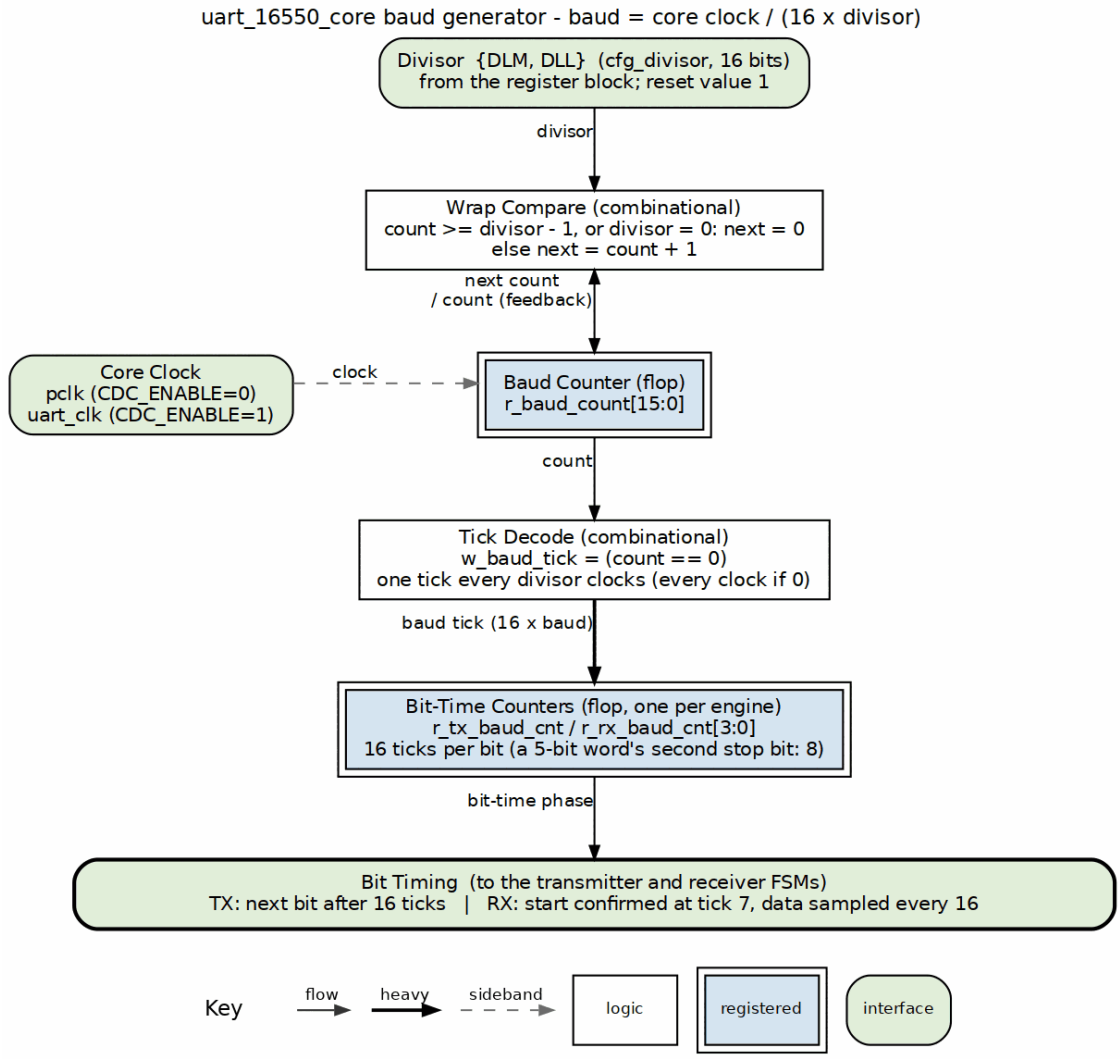
14 APB UART 16550 — Baud Generator Block

14.1 Overview

The baud generator turns your input clock into the 16x oversampled clock the TX and RX engines run on, divided down by the programmable {DLM, DLL} divisor. Get the divisor wrong and everything downstream is garbage — this is the one register pair you should double-check first.

14.2 Functional Description

14.2.1 Figure 2.5: Baud Generator Block



Baud Generator Block

14.2.2 Operation

14.2.2.1 Clock Division

Figure 2.5 shows the division: a 16-bit counter wraps at the {DLM, DLL} divisor and emits one baud tick per wrap, and each engine counts 16 ticks per bit.

14.2.2.2 Formula

```
16x_baud_clk = input_clk / divisor

where divisor = (DLM << 8) | DLL

Actual baud rate = 16x_baud_clk / 16 = input_clk / (16 * divisor)
```

14.2.3 Divisor Calculation

14.2.3.1 Standard Formula

```
Divisor = Input_Clock / (16 * Desired_Baud_Rate)
```

14.2.3.2 Rounding

For best accuracy, round to nearest integer:

```
Divisor = (Input_Clock + 8 * Baud_Rate) / (16 * Baud_Rate)
```

14.2.3.3 Example Tables

48 MHz Input Clock:

Baud Rate	Divisor	DLM	DLL	Actual Rate	Error
9600	312	0x01	0x38	9615.4	+0.16%
19200	156	0x00	0x9C	19230.8	+0.16%
38400	78	0x00	0x4E	38461.5	+0.16%
57600	52	0x00	0x34	57692.3	+0.16%
115200	26	0x00	0x1A	115384.6	+0.16%

50 MHz Input Clock:

Baud Rate	Divisor	DLM	DLL	Actual Rate	Error
9600	326	0x01	0x46	9585.9	-0.15%
19200	163	0x00	0xA3	19171.8	-0.15%
38400	81	0x00	0x51	38580.2	+0.47%

Baud Rate	Divisor	DLM	DLL	Actual Rate	Error
57600	54	0x00	0x36	57870.4	+0.47%
115200	27	0x00	0x1B	115740.7	+0.47%

14.2.4 Divisor Latch Registers

14.2.4.1 DLL (Divisor Latch LSB)

Address	0x24
Bits	[7:0]
Access	RW
Reset	0x01

14.2.4.2 DLM (Divisor Latch MSB)

Address	0x28
Bits	[7:0]
Access	RW
Reset	0x00

14.3 Usage Example

14.3.1 Programming Sequence

DLL/DLM have dedicated offsets (0x24/0x28) AND are reached at 0x00/0x04 while DLAB is set; the dedicated offsets remap no address, so no DLAB toggle is required.

1. Write DLL at 0x24 (divisor low byte)
2. Write DLM at 0x28 (divisor high byte)

```
void set_baud_rate(uint16_t divisor) {
    DLL = divisor & 0xFF;     // Low byte  (0x24)
    DLM = divisor >> 8;       // High byte (0x28)
}
```

14.4 Design Notes

14.4.1 Special Cases

14.4.1.1 Divisor = 0

- Invalid configuration; should be avoided

- In this RTL there is no divisor=0 guard: the baud tick asserts every clock (as if dividing by 1), so bit timing runs at the input clock rate

Note: DLL resets to 0x01, so the power-on divisor is 1 (not 0).

14.4.1.2 Divisor = 1

- Maximum baud rate
- Rate = input_clk / 16
- 48 MHz -> 3 Mbps

14.4.1.3 Clock Enable

The baud counter free-runs; there is no divisor!=0 or TX/RX-active gating in this RTL. The generated baud tick is used by the TX/RX engines when they are active.

14.5 Navigation

Next: 06_fifo.md - FIFO Subsystem

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

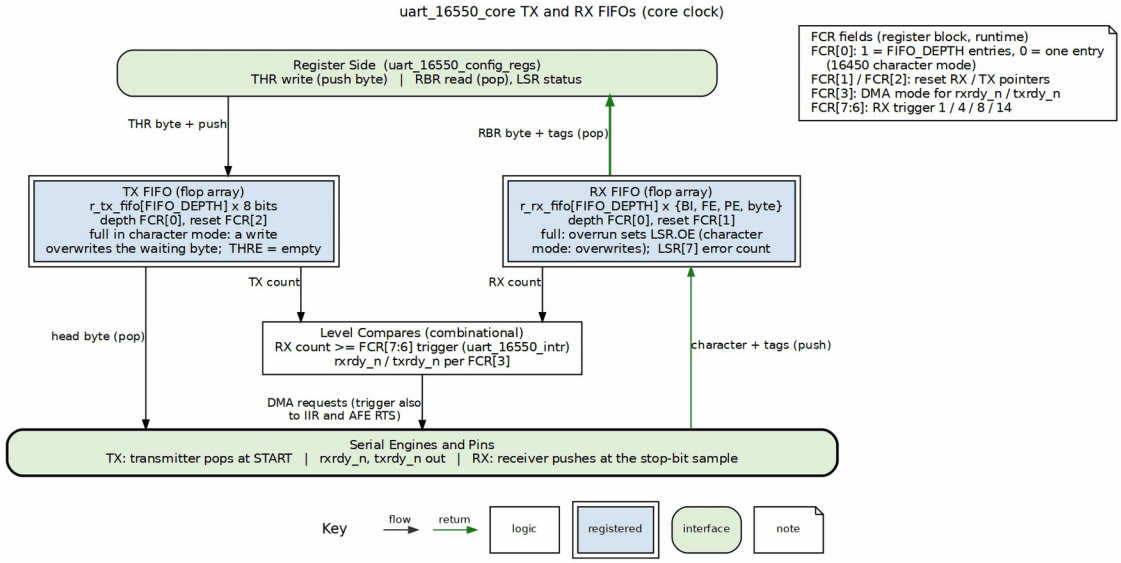
15 APB UART 16550 — FIFO Subsystem

15.1 Overview

Both directions get a 16-byte FIFO between software and the serial line, and the FCR register is how you manage them. One thing to internalize early: FCR[0] changes the buffering. With FCR[0]=1 each FIFO holds FIFO_DEPTH entries; with FCR[0]=0 each side is a single holding register (16450 character mode), a second THR write overwrites the waiting byte, and a second received character is an overrun.

15.2 Functional Description

15.2.1 Figure 2.6: FIFO Block



FIFO Block

15.2.2 FIFO Configuration

15.2.2.1 FCR (FIFO Control Register)

Bit	Name	Description
0	FE	FIFO Enable
1	RFR	RX FIFO Reset
2	TFR	TX FIFO Reset
3	DMS	DMA Mode Select: 0 = per character, 1 = per block, on rxrdy_n/txrdy_n
5:4	Reserved	

Bit	Name	Description
7:6	RTL	RX Trigger Level

15.2.2.2 Trigger Levels

RTL[1:0]	RX FIFO Trigger
00	1 byte
01	4 bytes
10	8 bytes
11	14 bytes

15.2.3 TX FIFO

15.2.3.1 Characteristics

Parameter	Value
Depth	16 bytes
Width	8 bits

15.2.3.2 Operations

Operation	Trigger
Write	THR register write
Read	TX serializer ready
Reset	FCR.TFR write

15.2.3.3 Status

Signal	LSR Bit	Condition
THRE	5	TX FIFO empty (`sts_tx_holding_empty = tx_fifo_empty`)
TEMT	6	FIFO empty AND shift register empty

15.2.4 RX FIFO

15.2.4.1 Characteristics

Parameter	Value
Depth	16 entries

Parameter	Value
Width	11 bits

15.2.4.2 Entry Format

```
[10] [9] [8] [7:0]
 BI  FE  PE  DATA
```

15.2.4.3 Operations

Operation	Trigger
Write	RX deserializer complete
Read	RBR register read
Reset	FCR.RFR write

15.2.4.4 Status

Signal	LSR Bit	Condition
DR	0	Data Ready (FIFO not empty)
OE	1	Overrun Error
PE	2	Parity Error (per character)
FE	3	Framing Error (per character)
BI	4	Break Indicator (per character)
FIFOERR	7	Error in the RX FIFO **head** entry (PE, FE, or BI) - not “any entry”

15.2.5 FIFO vs Non-FIFO Mode

15.2.5.1 FCR.FE = 0

- Both FIFOs remain physically 16 deep in this RTL (there is no true single-byte 8250 mode); THR writes still queue up to 16 bytes
- FCR.FE=0 only changes IIR[7:6] to 00 and makes the RX-data interrupt condition DR-based rather than trigger-level based

15.2.5.2 FCR.FE = 1 (16550 Mode)

- 16-byte FIFOs, trigger-level RX interrupt
- IIR[7:6] = 11
- (Character-timeout interrupt: four character times of inactivity, IIR 0x0C)

15.2.6 Error Handling

15.2.6.1 Per-Character Errors

Each RX FIFO entry carries its own PE, FE and BI tags in bits [10:8]. LSR[4:2] are loaded from the tags of the character the CPU is being handed, so an error is reported on the read that returns the character it belongs to, and LSR[7] aggregates the tags over the whole FIFO. Entry format: - Sticky LSR error flags are set at RECEIVE time (when the character enters the FIFO), not when it is later read out - LSR[7] reflects an error on the FIFO HEAD entry only, not ‘any entry’

15.2.6.2 Overrun Error

OE set immediately when: - RX FIFO full - New character received - New character discarded

15.2.7 FIFO Reset

15.2.7.1 TX FIFO Reset (FCR.TFR)

1. Write FCR with TFR=1
2. TX FIFO cleared immediately
3. The TX state machine is forced to IDLE - the character in progress is aborted
4. THRE and TEMT updated

15.2.7.2 RX FIFO Reset (FCR.RFR)

1. Write FCR with RFR=1
2. RX FIFO cleared immediately
3. The RX state machine is forced to IDLE - the character in progress is aborted
4. DR cleared

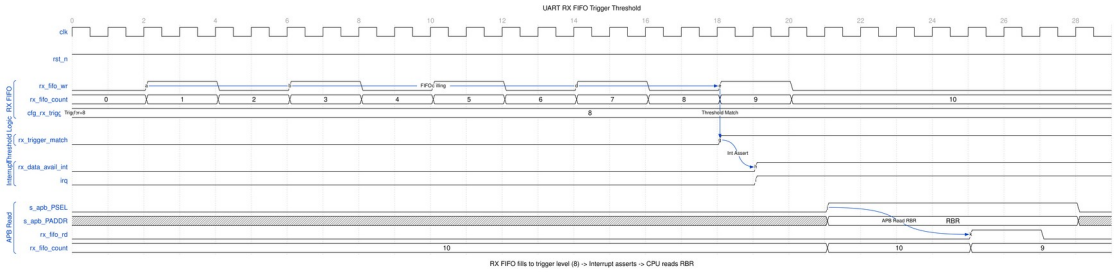
15.2.7.3 Full Reset

```
// Reset both FIFOs, enable with trigger=14
FCR = 0xC7;  // 11000111b
```

15.3 Waveforms

15.3.1 Waveform 2.5: FIFO Trigger Timing

The following diagram shows RX FIFO filling to the trigger threshold and generating an interrupt.



UART FIFO Threshold

The interrupt sequence: 1. RX data arrives, `rx_fifo_wr` pulses 2. FIFO count increments with each byte 3. When count reaches trigger level (8 in example), `rx_trigger_match` asserts 4. RX Data Available interrupt (`rx_data_avail_int`) triggers 5. CPU reads RBR to retrieve data, decrementing FIFO count

---

15.4 Navigation

Back to: 00_overview.md - Block Descriptions Overview

Next Chapter: Chapter 3: Interfaces

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

16 APB UART 16550 — Interfaces

16.1 Overview

Five external interfaces define how this block talks to the rest of your system: the APB slave port, the serial pins, the modem pins, the interrupt output, and the clock/reset inputs. This chapter specifies each one.

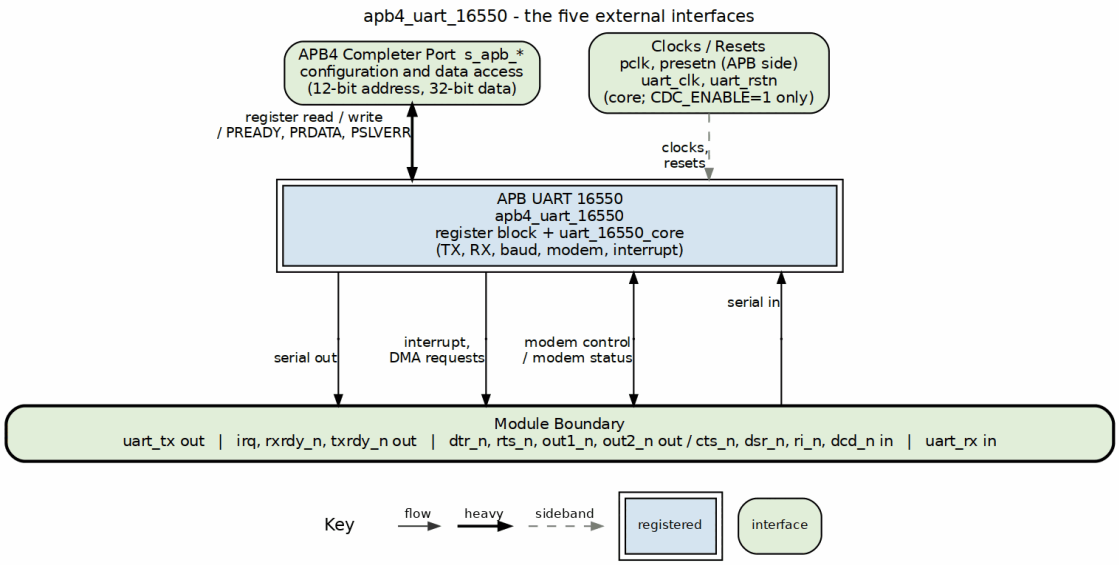
16.2 Ports

16.2.1 External Interfaces

Interface	Type	Description
APB Slave	Bus	Configuration and data access
Serial	I/O	TXD/RXD serial data
Modem	I/O	CTS, RTS, DTR, DSR, DCD, RI
Interrupt	Signal	IRQ output
Clocks/Reset	System	Clock and reset inputs

16.3 Functional Description

16.3.1 Figure 3.1: UART Interface Summary



UART Interfaces

16.4 Navigation

16.4.1 Chapter Contents

16.4.1.1 APB Slave Interface

Complete APB protocol interface for register access.

See: 01_apb4_slave.md

16.4.1.2 Serial Interface

TXD and RXD serial data connections.

See: 02_serial.md

16.4.1.3 Modem Interface

Hardware flow control and modem signals.

See: 03_modem.md

16.4.1.4 Interrupt Interface

Interrupt request output signal.

See: 04_interrupt.md

16.4.1.5 System Interface

Clock and reset signal requirements.

See: 05_system.md

Next: 01_apb4_slave.md - APB Slave Interface

---

17 APB UART 16550 — APB Slave Interface

17.1 Overview

This is the bus-level view of the block: the APB signals, the address map they reach, and the side effects each register access triggers. The side-effect table is the part worth memorizing — reads and writes here are not always free of consequences.

17.2 Ports

17.2.1 APB Slave Signals

Signal	Width	Dir	Description
pclk	1	I	APB clock
presetn	1	I	APB reset (active low)
s_apb_psel	1	I	Peripheral select
s_apb_penable	1	I	Enable phase
s_apb_pwrite	1	I	Write transaction
s_apb_paddr	12	I	Address bus
s_apb_pwdata	32	I	Write data
s_apb_pstrb	4	I	Byte strobes
s_apb_prdata	32	O	Read data
s_apb_pready	1	O	Ready response
s_apb_pslverr	1	O	Slave error

17.3 Functional Description

17.3.1 Address Map

Each register has a unique offset, and LCR[7] (DLAB) additionally remaps 0x00/0x04 to DLL/DLM while it is set; DLL/DLM keep their dedicated offsets either way. The decode is strict: only the mapped offsets, with `paddr[11:6]` zero, reach the register block, and any other address in the window reads 0 with PSLVERR; nothing aliases.

Offset	Read	Write
0x00	RBR	THR
0x04	IER	IER
0x08	IIR	-
0x0C	FCR	FCR
0x10	LCR	LCR
0x14	MCR	MCR
0x18	LSR	LSR (error bits clear on read)
0x1C	MSR	MSR (delta bits clear on read)
0x20	SCR	SCR
0x24	DLL	DLL
0x28	DLM	DLM

17.3.2 Protocol Compliance

17.3.2.1 APB3/APB4 Features

Feature	Support
PSEL	Yes
PENABLE	Yes
PWRITE	Yes
PADDR	12-bit
PWDATA	32-bit
PRDATA	32-bit
PREADY	Yes (inserts wait states – see Access Timing above)
PSLVERR	Yes - asserted on any unmapped offset in the window
PSTRB	Yes
PPROT	Present (s_apb_PPROT[2:0], accepted and unused)

17.3.3 Register Access

17.3.3.1 Byte Access

32-bit APB with 8-bit registers: - pstrb[0]: Access register at paddr - Other strobes: No effect (registers are 8-bit)

17.3.3.2 Side Effects

Some registers have read/write side effects:

Register	Read Side Effect	Write Side Effect
RBR	Pops RX FIFO	N/A
THR	N/A	Pushes TX FIFO
IIR	None (reading IIR does not clear any interrupt)	N/A
FCR	None (FCR is readable)	Can reset FIFOs
LSR	Clears error bits [4:1]	–
MSR	Clears delta bits [3:0]	–

Note: LSR error bits [4:1] and MSR delta bits [3:0] clear when their own register is read, as a standard 16550 does, and the line-status and modem-status interrupts deassert with them. Any read clears them, including one that was only checking data-ready, so software must work from the value the read returned.

17.4 Timing

17.4.1 Access Timing

PREADY inserts wait states: the apb4_slave bridge is a small FSM that issues the command and returns the response over a registered cmd/rsp handshake, so an access takes several pclk beyond the 2-cycle APB minimum (more when CDC_ENABLE=1). The register block itself never stalls.

17.5 Navigation

Next: 02_serial.md - Serial Interface

18 APB UART 16550 — Serial Interface

18.1 Overview

Two pins, `uart_tx` and `uart_rx`, carry everything. This page defines what those pins do electrically and logically — frame formats, sampling behavior, break handling — including the places where this RTL falls short of what the waveform diagrams were drawn to show.

18.2 Ports

18.2.1 Signal Description

Signal	Width	Dir	Description
uart_tx	1	O	Serial transmit data
uart_rx	1	I	Serial receive data

18.2.2 Electrical Interface

18.2.2.1 TTL Level

State	Voltage
Logic 0 (Space)	0V
Logic 1 (Mark)	VCC (3.3V/5V)

18.2.2.2 RS-232 Level (External Transceiver)

State	Voltage
Logic 0 (Space)	+3V to +15V
Logic 1 (Mark)	-3V to -15V

18.3 Functional Description

18.3.1 TXD (Transmit Data)

18.3.1.1 Characteristics

Parameter	Value
Idle State	Logic 1 (Mark)
Start Bit	Logic 0 (Space)
Stop Bit	Logic 1 (Mark)

Parameter	Value
Bit Order	LSB first

18.3.1.2 Frame Format

```
IDLE  START  D0  D1  D2  D3  D4  D5  D6  D7  PAR  STOP  IDLE
  1     0    |<-------- Data Bits -------->|  P    1     1
             |<-------- LSB first -------->|
```

18.3.1.3 Output Timing

Event	Timing
Bit transition	On baud clock edge
Bit duration	16 x (16x_clk period)
Start to first data	1 bit time

18.3.2 RXD (Receive Data)

18.3.2.1 Characteristics

Parameter	Value
Idle State	Logic 1 (Mark)
Break	Extended Logic 0
Sampling	Mid-bit (8th of 16 clocks)

18.3.2.2 Input Synchronization

```
RXD --> FF1 --> FF2 --> Synchronized RXD
       (clk)   (clk)
```

- Two-stage synchronizer
- Prevents metastability
- 2 clock cycle latency

18.3.2.3 Start Bit Detection

1. Detect falling edge (1 to 0)
2. Wait 8 clocks (half bit)
3. Verify still 0
4. Begin data sampling

18.3.3 Data Formats

18.3.3.1 Configurable Parameters (LCR)

Parameter	Options
Data bits	5, 6, 7, or 8
Stop bits	1 or 2 (no 1.5; and STB=1 with a 5-bit word still sends ONE stop bit)
Parity	None, Even, Odd, Mark, Space

18.3.3.2 Frame Examples

8N1 (8 data, No parity, 1 stop):

```
START | D0 D1 D2 D3 D4 D5 D6 D7 | STOP
  0   |<------ 8 bits -------->|  1
```

7E1 (7 data, Even parity, 1 stop):

```
START | D0 D1 D2 D3 D4 D5 D6 | EP | STOP
  0   |<---- 7 bits ------->|  P |  1
```

5N1 (5 data, No parity, 1 stop) – note 5N2 does not exist in this RTL (a 5-bit word always gets one stop bit regardless of STB):

```
START | D0 D1 D2 D3 D4 | STOP
  0   |<-- 5 bits -->|  1
```

18.3.4 Break Condition

18.3.4.1 Transmit Break

When LCR.BC=1: - TXD forced to Logic 0 - Maintained until BC cleared - Minimum duration: 1 frame time

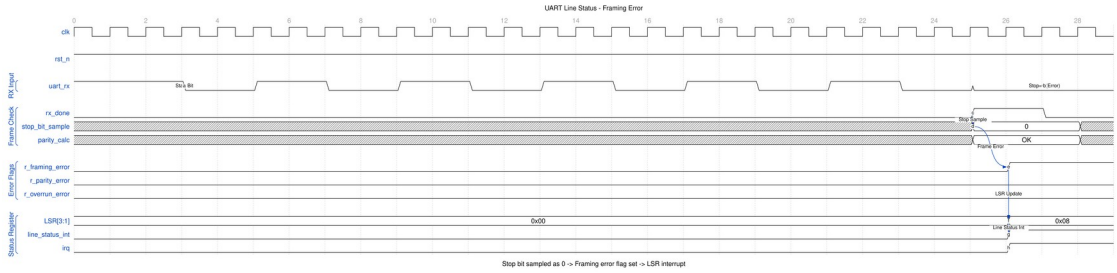
18.3.4.2 Receive Break

Detected when: - RXD = 0 for entire frame - Start + all data + parity + stop = 0 - Sets the BI bit in LSR and tags the FIFO entry. Exactly one zero character is loaded per break; the receiver then stays off until the line returns to marking and a genuine new start bit arrives

18.4 Waveforms

18.4.1 Waveform 3.1: Line Status Error Detection

The following diagram shows framing error detection when a stop bit is sampled as 0.



UART Line Status

Error detection sequence: 1. RX frame received normally (start, data bits) 2. Stop bit expected to be 1, but sampled as 0 3. Framing error flag set, and the character's FIFO entry tagged 4. LSR[3] (FE) reads 1 on the read that returns that character 5. Line status interrupt asserts, if enabled in IER[2]

Error types: - **Framing Error (FE)**: Stop bit not 1 - indicates baud rate mismatch or noise - **Parity Error (PE)**: Calculated vs received parity mismatch - **Overrun Error (OE)**: RX FIFO full when new character arrives - **Break Indicator (BI)**: All bits including stop are 0

18.5 Navigation

Next: 03_modem.md - Modem Interface

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

19 APB UART 16550 — Modem Interface

19.1 Overview

The modem interface carries the classic flow-control and status pins: RTS/DTR/OUT1/OUT2 out, CTS/DSR/DCD/RI in. Manage your expectations here — the automatic flow control you'd get from a real TL16C550C does not exist in this RTL, so software does the work.

19.2 Ports

19.2.1 Modem Control Outputs (Active Low)

Signal	Width	Dir	Description
rts_n	1	O	Request To Send
dtr_n	1	O	Data Terminal Ready
out1_n	1	O	User output 1
out2_n	1	O	User output 2

19.2.2 Modem Status Inputs (Active Low)

Signal	Width	Dir	Description
cts_n	1	I	Clear To Send
dsr_n	1	I	Data Set Ready
dcd_n	1	I	Data Carrier Detect
ri_n	1	I	Ring Indicator

19.3 Functional Description

19.3.1 Modem Control Register (MCR)

19.3.1.1 Output Control

Bit	Name	Signal	Active When
0	DTR	dtr_n	MCR[0] = 1
1	RTS	rts_n	MCR[1] = 1
2	OUT1	out1_n	MCR[2] = 1

Bit	Name	Signal	Active When
3	OUT2	out2_n	MCR[3] = 1 (also gates the irq pin)
4	LOOP	-	Loopback mode

MCR is 6 bits wide: bit 5 is AFE (auto flow control, see below). OUT2 additionally gates the `irq` output: the pin can assert only when MCR.OUT2 = 1.

19.3.1.2 Auto Flow Control (AFE)

MCR[5] enables auto flow control. With it set, the transmitter starts a character only while CTS is asserted - the character already in the shifter always finishes, AFE gates the start and not the frame - and RTS is driven from the RX FIFO level rather than from MCR[1]: it deasserts at the trigger level and reasserts once software has read the FIFO back below it. MCR[1] must still be set for RTS to be asserted at all. Leave AFE clear for manual flow control: drive MCR.RTS and check MSR.CTS in software.

19.3.2 Modem Status Register (MSR)

19.3.2.1 Current State (Read-Only)

Bit	Name	Source	Meaning
4	CTS	cts_n	Current CTS state
5	DSR	dsr_n	Current DSR state
6	RI	ri_n	Current RI state
7	DCD	dcd_n	Current DCD state

19.3.2.2 Delta Bits (Clear on Read)

These bits clear when MSR is read, and the modem-status interrupt deasserts with them. Any read clears them, so work from the value the read returned rather than reading MSR a second time.

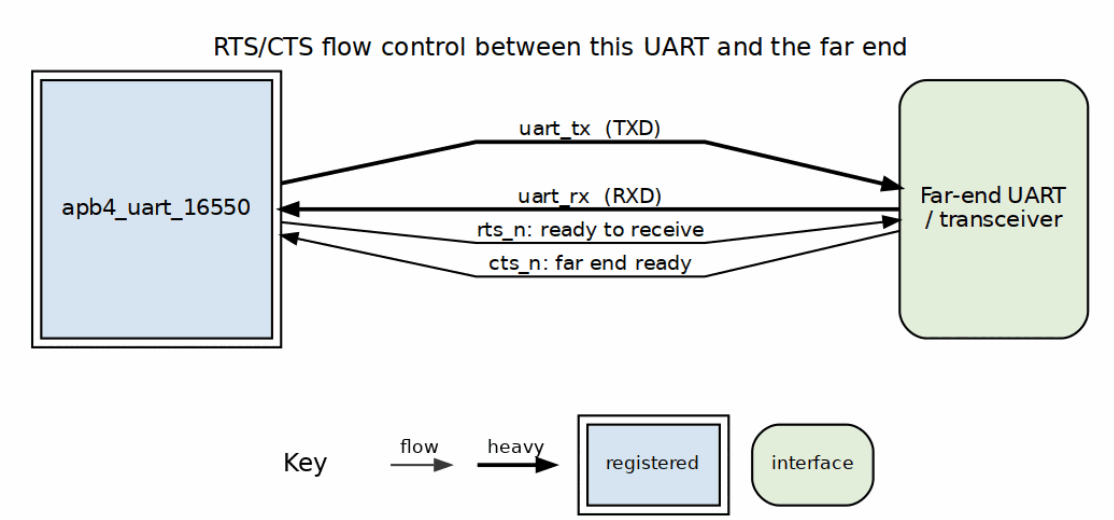
Bit	Name	Meaning
0	DCTS	CTS changed since last cleared
1	DDSR	DSR changed since

Bit	Name	Meaning
		last cleared
2	TERI	RI changed from low to high
3	DDCD	DCD changed since last cleared

19.3.3 Hardware Flow Control

19.3.3.1 RTS/CTS Flow Control

19.3.4 Figure 3.2: RTS/CTS Flow Control Connections



RTS/CTS Flow Control Connections

With AFE clear, RTS/CTS flow control is handled in software: 1. Software asserts MCR.RTS when ready to receive 2. Software checks MSR.CTS before sending 3. Hardware does not auto-pause TX on CTS

With AFE set, the hardware does all three.

19.3.4.1 Manual Flow Control

Software controls RTS directly:

```
// Ready to receive
MCR |= 0x02;   // Assert RTS

// Stop receiving
MCR &= ~0x02;  // Deassert RTS
```

19.3.5 Loopback Mode

When MCR.LOOP = 1: - TXD internally connected to RXD - Modem outputs connected to inputs: - DTR -> DSR - RTS -> CTS - OUT1 -> RI - OUT2 -> DCD - External signals disconnected

Used for: - Self-test - UART verification - Driver testing

19.3.6 Input Synchronization

All modem inputs pass through 2-stage synchronizer:

```
cts_n --> FF1 --> FF2 --> synced_cts_n
         (clk)   (clk)
```

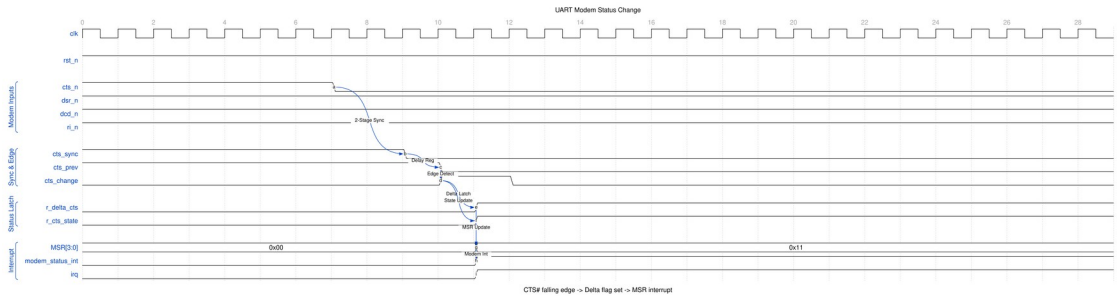
19.3.7 Interrupt Generation

MSR delta bits can generate the modem-status interrupt: - Any delta bit set generates the interrupt, gated by IER[3] - The `irq` pin is gated by MCR.OUT2 as well - Delta bits clear when MSR is read

19.4 Waveforms

19.4.1 Waveform 3.2: Modem Status Change Detection

The following diagram shows how modem input changes are detected and reported.



UART Modem Status

The detection sequence: 1. External CTS# falls (device ready to receive) 2. 2-stage synchronizer captures the change 3. Edge detector compares current vs. previous state 4. Delta flag (`r_delta_cts`) latched 5. Current state (`r_cts_state`) updated 6. MSR updated, modem status interrupt asserted

---

19.5 Navigation

Next: 04_interrupt.md - Interrupt Interface

20 APB UART 16550 — Interrupt Interface

20.1 Overview

Before you write a line of ISR code, read the notes below. This RTL's interrupt path does not behave like the 16550 your driver author probably had in mind — the enables don't enable, the timeout doesn't exist, and the pin is gated by a modem output bit.

> Implementation notes for this RTL: - **Each of the four sources is gated by its own IER bit.** A disabled source still sets its status bit; it simply is not an interrupt, does not raise `irq`, and is not reported by IIR. - **The `irq` pin is gated by MCR.OUT2** (`irq = pending & MCR.OUT2`). With the reset MCR = 0x00 the pin is masked; software must set OUT2 to route interrupts. - **The character timeout** fires after four character times with a non-empty RX FIFO and no activity; IIR reads 0x0C. It shares the received-data slot and is gated by IER[0]. - **Reading IIR clears the THR-empty interrupt** when THR empty is the source it reported, as a standard 16550 does. - LSR[4:1] clear on a read of LSR, and MSR[3:0] on a read of MSR.

20.2 Ports

20.2.1 Signal Description

Signal	Width	Dir	Description
irq	1	O	Interrupt request (active high, gated by MCR.OUT2)

20.3 Functional Description

20.3.1 Interrupt Sources

20.3.1.1 Priority Order (Highest to Lowest)

Priority	IIR[3:0]	Source	Clear Method
1	0110	Receiver Line Status	Read LSR
2	0100	Received Data Available	Read RBR until DR clears

Priority	IIR[3:0]	Source	Clear Method
3	0010	THR Empty	Write THR (fill TX FIFO)
4	0000	Modem Status	Read MSR

Character Timeout reads IIR = 1100 (0x0C) and shares priority 2 with received-data-available.

20.3.1.2 IIR Encoding

IIR[3:0]	IIR[0]	Meaning
xxx0	0	Interrupt pending
xxx1	1	No interrupt

20.3.2 Interrupt Enable Register (IER)

These bits are stored and read back but are **not connected** to the interrupt logic in this RTL - they do not enable or mask any interrupt.

Bit	Name	Nominal Source (not enforced)
0	ERBFI	Received data available
1	ETBEI	THR empty
2	ELSI	Receiver line status
3	EDSSI	Modem status

20.3.3 Interrupt Identification Register (IIR)

20.3.3.1 Read Format

Bits	Name	Description
0	IPEND	0=interrupt pending, 1=none
3:1	IID	Interrupt ID
5:4	Reserved	
7:6	FIFOEN	11 if FIFOs enabled

20.3.4 Interrupt Conditions

20.3.4.1 Receiver Line Status (Priority 1)

Triggered by: - Overrun Error (OE) - Parity Error (PE) - Framing Error (FE) - Break Indicator (BI)

Cleared by reading LSR.

20.3.4.2 Received Data Available (Priority 2)

- FCR.FE=1: triggered when RX FIFO >= trigger level
- FCR.FE=0: triggered when data present (DR=1)
- Cleared by reading RBR until the level falls below the trigger / DR clears

20.3.4.3 Character Timeout (Priority 2)

Four character times with the RX FIFO non-empty and neither a new character nor a read. IIR reads 0x0C. Any FIFO activity restarts the timer, and the source exists only in FIFO mode.

20.3.4.4 THR Empty (Priority 3)

Triggered when the TX FIFO is empty (this is a level, THRE = TX FIFO empty).

Cleared by writing THR to refill the TX FIFO. Reading IIR does **not** clear it.

20.3.4.5 Modem Status (Priority 4)

Triggered by any MSR delta bit: - DCTS (Delta CTS) - DDSR (Delta DSR) - TERI (Trailing Edge RI) - DDCD (Delta DCD)

Cleared by reading MSR.

20.4 Timing

20.4.1 Interrupt Timing

IIR and `irq` are purely COMBINATIONAL from the core status flags: the cycle a condition's flag sets, IIR reflects it and `irq` asserts (gated by MCR.OUT2); the cycle the condition clears, both deassert. There are no pipeline stages in the interrupt path.

```
Event --> flag sets --> IIR + IRQ reflect it (same cycle,
combinational)
Clear --> flag clears --> IIR + IRQ deassert (same cycle)
```

20.5 Usage Example

20.5.1 Software Handling

20.5.1.1 ISR Flow

```c
void uart_isr(void) {
    uint8_t iir = IIR;

    while ((iir & 0x01) == 0) {  // While interrupt pending
        switch (iir & 0x0E) {
```

```
            case 0x06:  // Line status
                handle_line_status(LSR);
                break;
            case 0x04:  // RX data available
            case 0x0C:  // Character timeout
                handle_rx_data();
                break;
            case 0x02:  // THR empty
                handle_tx_empty();
                break;
            case 0x00:  // Modem status
                handle_modem_status(MSR);
                break;
        }
        iir = IIR;  // Re-read for next pending
    }
}
```

20.6 Navigation

Next: 05_system.md - System Interface

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

21 APB UART 16550 — System Interface

21.1 Overview

The system-level plumbing: clocks, resets, what everything resets to, and what the block looks like from the outside when you drop it into a design. The power-management section is short — because the block gives you nothing to work with there.

21.2 Ports

21.2.1 Clock Signals

21.2.1.1 pclk - APB Clock

Parameter	Value
Purpose	APB interface and UART logic
Frequency	50-200 MHz typical
Domain	All internal logic

21.2.1.2 Baud Rate Derivation

Baud rate is derived from pclk:

```
Baud Rate = pclk / (16 * Divisor)
```

21.2.2 Reset Signals

21.2.2.1 presetn - APB Reset

Parameter	Value
Polarity	Active low
Type	Asynchronous assert, synchronous deassert (hand-written logic; the generated register file resets synchronously)
Scope	All UART logic

21.3 Functional Description

21.3.1 Reset Behavior

21.3.1.1 Register Reset Values

Register	Reset	Notes
RBR	Undefined	FIFO content
THR	N/A	Write-only
IER	0x00	Interrupt enables, all four sources
IIR	0x02	THR-empty pending at reset
FCR	0x00	FIFO-enable interface bit clear
LCR	0x03	8N1 format
MCR	0x00	Outputs deasserted (irq masked - OUT2=0)
LSR	0x60	TX empty
MSR	0x00	Inputs low
SCR	0x00	Cleared
DLL	0x01	Divisor LSB = 1
DLM	0x00	Divisor MSB = 0

21.3.1.2 Signal States During Reset

Signal	Reset State
uart_tx	1 (Mark/Idle)
rts_n	1 (Deasserted)
dtr_n	1 (Deasserted)
out1_n	1 (Deasserted)
out2_n	1 (Deasserted)
irq	0 (No interrupt)

21.3.1.3 Post-Reset Initialization

1. Set baud rate (write DLL at 0x24, DLM at 0x28 directly - no DLAB toggle)

2. Configure line format (LCR at 0x10)
3. Enable FIFOs if desired (FCR at 0x0C)
4. Configure modem control (MCR at 0x14); set OUT2 to enable the irq pin
5. (IER at 0x04 gates each of the four interrupt sources independently)

21.4 Timing

21.4.1 Reset Sequence

21.4.2 Figure 3.3: Reset Sequence

UART 16550 reset sequence (CDC_ENABLE=0: one clock, pclk)

step	1	2	3
phase	**reset active**	**recovery**	**normal operation**
presetn	0	**1**	
hold for	>= 2 pclk cycles	2 pclk cycles	-
pclk	running		
outputs	reset states (uart_tx=1, irq=0)	reset states	per registers
APB access	not allowed		allowed

pclk must run during the hold: the generated register file resets synchronously, on a clock edge. Program the divisor before use. CDC_ENABLE=1: assert uart_rstn together with presetn (or quiesce the APB side first); a one-sided reset during a transfer corrupts the async FIFOs.

Reset Sequence

21.4.2.1 Requirements

- Hold reset low for minimum 2 pclk cycles
- Allow 2 cycles after reset before first APB access
- Divisor must be programmed before operation

21.5 Design Notes

21.5.1 Power Management

21.5.1.1 Clock Gating

The block provides NO gating or wake hooks: there are no power-management ports, and the baud counter free-runs unconditionally. Any clock gating is the integrator's, done OUTSIDE the

block – and gating pclk kills the APB interface while gating uart_clk kills RX sampling. There is no wake-on-activity path.

21.5.1.2 Low Power Hints

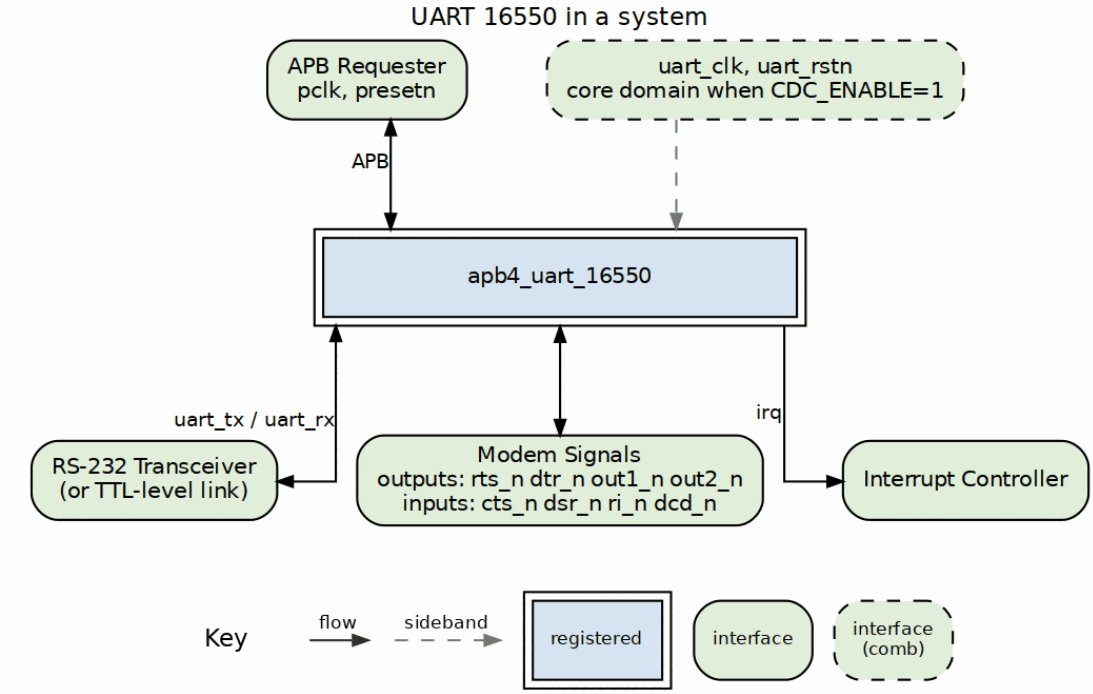
- Use FIFO mode to reduce interrupt rate
- Set a higher RX trigger level to reduce interrupt frequency

(Set MCR[5] for auto flow control, which paces the far end from the RX FIFO level.)

21.5.2 External Connections

21.5.2.1 Typical System

21.5.3 Figure 3.4: UART 16550 in a System



UART 16550 in a System

21.5.3.1 Direct Connection (TTL)

For TTL-level serial: - Connect uart_tx/uart_rx directly to 3.3V/5V logic - No level conversion needed - Short cable runs recommended

---

21.6 Navigation

Back to: 00_overview.md - Interfaces Overview

Next Chapter: Chapter 4: Programming Model

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

22 APB UART 16550 — Programming Model

22.1 Overview

This chapter is the software view of the block: how to initialize it, move data through it, and live with its interrupt behavior. The register summary below is the quick reference; the quick-start snippet will get characters on the wire.

22.1.1 Register Summary

Each register has a unique offset, and DLAB additionally remaps 0x00/0x04 to the divisor latches.

Offset	Register	Access	Description
0x00	RBR / THR	R / W	Receive Buffer (read) / Transmit Holding (write)
0x04	IER	RW	Interrupt Enable
0x08	IIR	RO	Interrupt Identification
0x0C	FCR	RW	FIFO Control
0x10	LCR	RW	Line Control
0x14	MCR	RW	Modem Control
0x18	LSR	RO, clear on read	Line Status
0x1C	MSR	RO, clear on read	Modem Status
0x20	SCR	RW	Scratch
0x24	DLL	RW	Divisor Latch LSB
0x28	DLM	RW	Divisor Latch MSB

22.2 Usage Example

22.2.1 Quick Start

22.2.1.1 Minimal Setup (115200 8N1)

```
// Assuming 48 MHz clock
#define DIVISOR 26  // 48MHz / (16 * 115200) = 26

void uart_init(void) {
    // Set baud rate divisor directly - no DLAB toggle (DLL=0x24,
DLM=0x28)
```

```
    DLL = DIVISOR & 0xFF;
    DLM = DIVISOR >> 8;

    // 8N1
    LCR = 0x03;

    // Enable FIFOs, reset, trigger=14
    FCR = 0xC7;

    // Set MCR.OUT2 to ungate the irq pin. It gates the pin in
addition to
    // IER, and resets to 0, so interrupts stay masked until this is
set.
    MCR = 0x08;
}
```

22.3 Navigation

22.3.1 Chapter Contents

22.3.1.1 Initialization

Complete UART initialization sequence.

See: 01_initialization.md

22.3.1.2 Data Transfer

Sending and receiving data.

See: 02_data_transfer.md

22.3.1.3 Interrupts

Interrupt configuration and handling.

See: 03_interrupts.md

22.3.1.4 Examples

Complete programming examples.

See: 04_examples.md

Next: 01_initialization.md - Initialization

23 APB UART 16550 — Initialization

23.1 Overview

Four steps stand between reset and a working serial link: baud rate, line format, FIFOs, and interrupts. Each step below is short, and each has at least one place where this RTL differs from a textbook 16550 — the comments call those out.

23.2 Functional Description

23.2.1 Baud Rate Calculation

23.2.1.1 Formula

```
Divisor = Clock_Frequency / (16 * Baud_Rate)
```

23.2.1.2 Divisor Calculator Function

```
uint16_t uart_calculate_divisor(uint32_t clock_hz, uint32_t baud) {
    // Round to nearest
    return (clock_hz + (8 * baud)) / (16 * baud);
}
```

23.2.1.3 Common Clock Frequencies

Clock	9600	19200	38400	57600	115200
48 MHz	312	156	78	52	26
50 MHz	326	163	81	54	27
100 MHz	651	326	163	109	54

23.3 Usage Example

23.3.1 Basic Initialization Sequence

23.3.1.1 Step 1: Set Baud Rate

```
void uart_set_baud(uint16_t divisor) {
    // DLL (0x24) and DLM (0x28) have dedicated offsets - no DLAB
toggle needed.
    DLL = divisor & 0xFF;
    DLM = (divisor >> 8) & 0xFF;
}
```

23.3.1.2 Step 2: Configure Line Format

```
void uart_set_format(uint8_t data_bits, uint8_t parity, uint8_t
stop_bits) {
```

```
    uint8_t lcr = 0;

    // Data bits: 5=0, 6=1, 7=2, 8=3
    lcr |= (data_bits - 5) & 0x03;

    // Stop bits: 1=0, 2=1. STB=1 gives 2 stop bits for 6/7/8-bit
words.
    // 1.5 stop bits: a 5-bit word with LCR[2] set sends a half-length
second stop bit.
    if (stop_bits == 2) lcr |= 0x04;

    // Parity: 0=none, 1=odd, 2=even, 3=mark, 4=space
    switch (parity) {
        case 1: lcr |= 0x08; break;        // Odd
        case 2: lcr |= 0x18; break;        // Even
        case 3: lcr |= 0x28; break;        // Mark
        case 4: lcr |= 0x38; break;        // Space
    }

    LCR = lcr;
}
```

23.3.1.3 Step 3: Configure FIFOs

```
void uart_configure_fifo(uint8_t trigger_level) {
    uint8_t fcr = 0x01;  // Enable FIFOs

    // Trigger level: 0=1, 1=4, 2=8, 3=14
    fcr |= (trigger_level & 0x03) << 6;

    // Reset both FIFOs
    fcr |= 0x06;

    FCR = fcr;
}
```

23.3.1.4 Step 4: Enable Interrupts

```
void uart_enable_interrupts(uint8_t mask) {
    // Bits: 0=RX, 1=TX, 2=Line, 3=Modem
    // Each bit gates its own source. The irq pin is gated by MCR.OUT2
as
    // well, so set OUT2 before expecting an interrupt.
    IER = mask & 0x0F;
}
```

23.3.2 Complete Initialization Example

```
// Common baud rate divisors for 48 MHz clock
#define BAUD_9600    312
#define BAUD_19200   156
#define BAUD_38400   78
#define BAUD_57600   52
#define BAUD_115200  26

void uart_init_115200_8n1(void) {
    // 1. Set baud rate (DLL=0x24, DLM=0x28 - no DLAB toggle)
    DLL = BAUD_115200;       // Low byte
    DLM = 0x00;              // High byte

    // 2. Line format: 8 data bits, no parity, 1 stop bit
    LCR = 0x03;

    // 3. Enable and reset FIFOs, trigger at 14 bytes
    FCR = 0xC7;              // 11000111b

    // 4. Modem control: set OUT2 (bit 3) to ungate the irq pin
    MCR = 0x08;

    // 5. Drain any pending RX data (received byte is in RBR bits
[7:0])
    while (LSR & 0x01)
        (void)RBR;

    // NOTE: LSR[4:1] and MSR[3:0] clear when their own register is
read,
    // so the drain loop above consumes any error bits it passes over.
}
```

23.3.3 Flow Control Setup (manual)

Auto flow control (AFE, MCR[5]) is implemented. For manual control leave AFE clear and assert RTS in software and monitor MSR.CTS to gate transmission manually.

```
void uart_assert_rts(void) {
    uint8_t mcr = MCR;
    mcr |= 0x02;             // RTS (bit 1); set bit 5 as well for
auto flow control
    MCR = mcr;
}
```

23.3.4 Loopback Mode Setup

```
void uart_enable_loopback(void) {
    uint8_t mcr = MCR;
    mcr |= 0x10;              // LOOP bit
    MCR = mcr;
}
```

23.4 Navigation

Next: 02_data_transfer.md - Data Transfer

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

24 APB UART 16550 — Data Transfer

24.1 Overview

Moving bytes in and out of this UART is ordinary 16550 fare with three local twists, summarized below and repeated wherever they bite. The examples in this page already account for all three.

> Implementation notes for this RTL: reading offset 0x00 (RBR) returns the received byte in bits **[7:0]** and pops the RX FIFO; LSR[4:1] and MSR[3:0] clear on a read of their own register, so a poll of LSR consumes the error bits it reports; and `irq` is gated by MCR.OUT2 as well as by IER, so interrupt-driven code must set OUT2 = 1. See Chapter 5 for detail.

24.2 Usage Example

24.2.1 Transmitting Data

24.2.1.1 Polling Mode

```
void uart_putchar(uint8_t c) {
    // Wait for THR empty
    while ((LSR & 0x20) == 0);

    // Write data
    THR = c;
}

void uart_puts(const char *s) {
    while (*s) {
        uart_putchar(*s++);
    }
}
```

24.2.1.2 Interrupt-Driven TX

```
volatile uint8_t tx_buffer[256];
volatile uint8_t tx_head = 0;
volatile uint8_t tx_tail = 0;

void uart_send(const uint8_t *data, size_t len) {
    for (size_t i = 0; i < len; i++) {
        // Add to buffer
        tx_buffer[tx_head++] = data[i];
    }

    // Enable TX interrupt
```

```
    IER |= 0x02;
}

// In ISR when THRE interrupt:
void uart_tx_isr(void) {
    while ((LSR & 0x20) && (tx_head != tx_tail)) {
        THR = tx_buffer[tx_tail++];
    }

    if (tx_head == tx_tail) {
        IER &= ~0x02;  // Disable TX interrupt
    }
}
```

24.2.1.3 Waiting for TX Complete

```
void uart_flush(void) {
    // Wait for TEMT (both FIFO and shift register empty)
    while ((LSR & 0x40) == 0);
}
```

24.2.2 Receiving Data

24.2.2.1 Polling Mode

```
int uart_getchar(void) {
    // Check for data available
    if ((LSR & 0x01) == 0) {
        return -1;  // No data
    }

    // Read data (received byte is in [7:0]
    // chapter header note)
    return RBR & 0xFF;
}

int uart_getchar_blocking(void) {
    // Wait for data
    while ((LSR & 0x01) == 0);

    return RBR & 0xFF;
}
```

24.2.2.2 Interrupt-Driven RX

```
volatile uint8_t rx_buffer[256];
volatile uint8_t rx_head = 0;
volatile uint8_t rx_tail = 0;
```

```
// In ISR when RX data available:
void uart_rx_isr(void) {
    while (LSR & 0x01) {
        rx_buffer[rx_head++] = RBR & 0xFF;
    }
}

int uart_read(uint8_t *data, size_t max_len) {
    size_t count = 0;
    while ((rx_head != rx_tail) && (count < max_len)) {
        data[count++] = rx_buffer[rx_tail++];
    }
    return count;
}
```

24.2.3 Error Handling

24.2.3.1 Checking Line Status

```
uint8_t uart_check_errors(void) {
    uint8_t lsr = LSR;
    uint8_t errors = 0;

    if (lsr & 0x02) errors |= ERR_OVERRUN;
    if (lsr & 0x04) errors |= ERR_PARITY;
    if (lsr & 0x08) errors |= ERR_FRAMING;
    if (lsr & 0x10) errors |= ERR_BREAK;

    return errors;
}
```

24.2.3.2 Handling Errors in RX ISR

```
void uart_rx_error_isr(void) {
    uint8_t lsr = LSR;  // Read status once; the read clears the error
bits

    if (lsr & 0x02) {
        // Overrun - data lost
        stats.overrun_count++;
    }
    if (lsr & 0x04) {
        // Parity error
        stats.parity_count++;
    }
    if (lsr & 0x08) {
        // Framing error
        stats.framing_count++;
    }
```

```
    if (lsr & 0x10) {
        // Break received
        handle_break_condition();
    }
}
```

24.2.4 FIFO Management

24.2.4.1 FIFO Status

```
bool uart_tx_fifo_empty(void) {
    return (LSR & 0x20) != 0;  // THRE
}

bool uart_tx_complete(void) {
    return (LSR & 0x40) != 0;  // TEMT
}

bool uart_rx_data_ready(void) {
    return (LSR & 0x01) != 0;  // DR
}

bool uart_rx_fifo_error(void) {
    return (LSR & 0x80) != 0;  // FIFO error
}
```

24.2.4.2 FIFO Reset

```
void uart_reset_fifos(void) {
    // Reset both FIFOs while keeping enabled
    uint8_t fcr = FCR;
    FCR = fcr | 0x06;  // RFR + TFR
}
```

24.2.5 Bulk Transfer

24.2.5.1 Efficient TX (FIFO-aware)

```
size_t uart_write(const uint8_t *data, size_t len) {
    size_t sent = 0;

    while (sent < len) {
        // Wait for FIFO space
        if (LSR & 0x20) {
            // Fill FIFO (up to 16 bytes)
            size_t chunk = (len - sent < 16) ? (len - sent) : 16;
            for (size_t i = 0; i < chunk; i++) {
                THR = data[sent++];
            }
        }
```

```
    }

    return sent;
}
```

24.2.5.2 Efficient RX (FIFO-aware)

```
size_t uart_read_available(uint8_t *data, size_t max_len) {
    size_t count = 0;

    while ((LSR & 0x01) && (count < max_len)) {
        data[count++] = RBR & 0xFF;  // received byte is at [7:0]
    }

    return count;
}
```

24.3 Navigation

Next: 03_interrupts.md - Interrupts

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

25 APB UART 16550 — Interrupt Handling

25.1 Overview

Interrupt handling follows the 16550 datasheet, with one gap. The note below is the contract; everything else on this page follows from it.

> Implementation note: each of the four sources is gated by its own IER bit, and `irq` is additionally gated by MCR.OUT2 (set OUT2 = 1 to route the pin). LSR[4:1] clear on a read of LSR and MSR[3:0] on a read of MSR; reading IIR clears the THR-empty interrupt when that is the source it reported. The character timeout shares the received-data slot, reads IIR = 0x0C and is gated by IER[0].

25.2 Functional Description

25.2.1 Interrupt Identification Register (IIR)

25.2.1.1 IIR Values

Value	Priority	Interrupt Source	Clear Method
0x01	-	No interrupt	-
0x06	1	Line status error	Read LSR to clear the error bits
0x04	2	RX data available	Read RBR until DR clears
0x02	3	THR empty	Write THR (fills TX FIFO)
0x00	4	Modem status	Read MSR to clear the delta bits

Note: Character timeout (IIR = 0x0C) fires after four character times of inactivity with a non-empty RX FIFO. Reading IIR has no side effect (it does not clear the THR-empty condition).

25.3 Usage Example

25.3.1 Interrupt Enable Register (IER)

```
// Enable specific interrupts
#define IER_RDA     0x01    // Received Data Available
#define IER_THRE    0x02    // THR Empty
#define IER_RLS     0x04    // Receiver Line Status
#define IER_MS      0x08    // Modem Status

void uart_enable_rx_interrupt(void) {
    IER |= IER_RDA;
}

void uart_enable_tx_interrupt(void) {
    IER |= IER_THRE;
}

void uart_disable_tx_interrupt(void) {
    IER &= ~IER_THRE;
}
```

25.3.2 Complete ISR Example

```
void uart_isr(void) {
    uint8_t iir;

    // Loop while interrupts pending
    while (((iir = IIR) & 0x01) == 0) {
        switch (iir & 0x0E) {
            case 0x06:  // Receiver Line Status (highest priority)
                uart_handle_line_status();
                break;

            case 0x04:  // Received Data Available
                uart_handle_rx_data();
                break;

            case 0x0C:  // Character Timeout
                uart_handle_timeout();
                break;

            case 0x02:  // THR Empty
                uart_handle_tx_empty();
                break;

            case 0x00:  // Modem Status (lowest priority)
```

```
                uart_handle_modem_status();
                break;
        }
    }
}
```

25.3.3 Individual Interrupt Handlers

25.3.3.1 Line Status Handler

```
void uart_handle_line_status(void) {
    uint8_t lsr = LSR;  // Read status; this read also clears the
error bits

    if (lsr & 0x02) {
        // Overrun Error - FIFO overflow
        stats.overrun++;
    }
    if (lsr & 0x04) {
        // Parity Error
        stats.parity_err++;
    }
    if (lsr & 0x08) {
        // Framing Error
        stats.framing_err++;
    }
    if (lsr & 0x10) {
        // Break Indicator
        handle_break();
    }

    // Nothing to write back: LSR[4:1] were cleared by the read above.
    // That is also why this handler must work from `lsr`, the value
the
    // read returned, and must not read LSR again.
}
```

25.3.3.2 RX Data Handler

```
void uart_handle_rx_data(void) {
    // Read all available data from FIFO
    while (LSR & 0x01) {
        uint8_t data = RBR & 0xFF;  // received byte is at [7:0]
        rx_buffer[rx_head++] = data;

        if (rx_head >= RX_BUFFER_SIZE) {
            rx_head = 0;
        }
    }
```

```
    // Signal waiting thread/task
    signal_rx_available();
}
```

25.3.3.3 Character Timeout Handler

```
// Called when IIR reads 0x0C: four character times with data sitting
in
// the RX FIFO below the trigger level.
void uart_handle_timeout(void) {
    // Same as RX data - flush remaining FIFO data
    uart_handle_rx_data();

    // May want to signal "end of packet" condition
    signal_rx_timeout();
}
```

25.3.3.4 TX Empty Handler

```
void uart_handle_tx_empty(void) {
    // Fill TX FIFO from buffer
    while ((LSR & 0x20) && (tx_tail != tx_head)) {
        THR = tx_buffer[tx_tail++];

        if (tx_tail >= TX_BUFFER_SIZE) {
            tx_tail = 0;
        }
    }

    // If buffer empty, disable TX interrupt
    if (tx_tail == tx_head) {
        IER &= ~IER_THRE;
        signal_tx_complete();
    }
}
```

25.3.3.5 Modem Status Handler

```
void uart_handle_modem_status(void) {
    uint8_t msr = MSR;  // Read status; this read also clears the
delta bits

    if (msr & 0x01) {   // Delta CTS
        // CTS changed - update flow control
    }
    if (msr & 0x02) {   // Delta DSR
        // DSR changed - device status
    }
```

```
    if (msr & 0x04) {   // Trailing Edge RI
        // Ring detected
    }
    if (msr & 0x08) {   // Delta DCD
        // Carrier changed - connection status
    }

    // Nothing to write back: MSR[3:0] were cleared by the read above,
    // which is why this handler works from `msr` and does not re-
read.
}
```

25.3.4 Disabling/Enabling Interrupts

`IER = 0x00` disables all four sources. MCR.OUT2 gates the pin as well, so clearing it masks the pin whatever IER says:

```
// Mask the irq pin via the OUT2 gate
uint8_t saved_mcr = MCR;
MCR = saved_mcr & ~0x08;   // OUT2 = 0 -> irq pin held deasserted

// ... critical section ...

// Restore
MCR = saved_mcr;
```

25.4 Design Notes

25.4.1 Interrupt Latency Considerations

25.4.1.1 Trigger Level Selection

Trigger	Bytes in FIFO	Latency Budget	Best For
1	1	1 char time	Low latency
4	4	4 char times	Balanced
8	8	8 char times	Higher rates
14	14	2 char times*	Maximum efficiency

*Only 2 characters before overflow at 16-byte FIFO

25.4.1.2 Character Timeout

- Implemented: four character times with a non-empty RX FIFO and no activity, IIR reads 0x0C.

- For variable-length packets, poll LSR.DR and apply a software inactivity timeout instead.

25.5 Navigation

Next: 04_examples.md - Examples

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

26 APB UART 16550 — Programming Examples

26.1 Overview

Complete, working examples that pull together everything from the previous pages: a polled debug console, an interrupt-driven ring buffer, a small command-line interface, a loopback self-test, and manual flow control. All of them already account for this RTL's quirks — RBR at [7:0], no DLAB remap — so you can lift them directly.

26.2 Usage Example

26.2.1 Debug Console

26.2.1.1 Simple Polling Implementation

```
#define UART_BASE   0xFEC08000

// Each register has its own offset; this example uses those flat
offsets.
// (LCR[7]/DLAB additionally remaps 0x00/0x04 to DLL/DLM when set.)
// RBR read returns the received byte in bits [7:0], so read RBR as
16-bit.
#define RBR  (*(volatile uint16_t *)(UART_BASE + 0x00))
#define THR  (*(volatile uint8_t  *)(UART_BASE + 0x00))
#define IER  (*(volatile uint8_t  *)(UART_BASE + 0x04))
#define IIR  (*(volatile uint8_t  *)(UART_BASE + 0x08))
#define FCR  (*(volatile uint8_t  *)(UART_BASE + 0x0C))
#define LCR  (*(volatile uint8_t  *)(UART_BASE + 0x10))
#define MCR  (*(volatile uint8_t  *)(UART_BASE + 0x14))
#define LSR  (*(volatile uint8_t  *)(UART_BASE + 0x18))
#define MSR  (*(volatile uint8_t  *)(UART_BASE + 0x1C))
#define SCR  (*(volatile uint8_t  *)(UART_BASE + 0x20))
#define DLL  (*(volatile uint8_t  *)(UART_BASE + 0x24))
#define DLM  (*(volatile uint8_t  *)(UART_BASE + 0x28))

// Received byte is in RBR bits [7:0].
#define RBR_BYTE()  ((uint8_t)(RBR & 0xFF))

void debug_uart_init(void) {
    // 115200 baud, 8N1. No DLAB toggle - DLL/DLM have their own
offsets.
    DLL = 26;             // 48MHz / (16 * 115200)
    DLM = 0;
    LCR = 0x03;           // 8N1
    FCR = 0x07;           // Enable FIFOs, reset
    // IER left at 0: this console polls LSR rather than taking
```

```
interrupts.
}

void debug_putchar(char c) {
    while ((LSR & 0x20) == 0);
    THR = c;
}

void debug_puts(const char *s) {
    while (*s) {
        if (*s == '\n')
            debug_putchar('\r');
        debug_putchar(*s++);
    }
}

int debug_getchar(void) {
    if (LSR & 0x01)
        return RBR_BYTE();   // received byte is in RBR bits [7:0]
    return -1;
}
```

26.2.2 Ring Buffer Implementation

```
#define RX_BUF_SIZE 256
#define TX_BUF_SIZE 256

typedef struct {
    volatile uint8_t buffer[RX_BUF_SIZE];
    volatile uint16_t head;
    volatile uint16_t tail;
} ring_buffer_t;

static ring_buffer_t rx_buf;
static ring_buffer_t tx_buf;

void uart_isr(void) {
    uint8_t iir;

    while (((iir = IIR) & 0x01) == 0) {
        switch (iir & 0x0E) {
            case 0x0C:  // character timeout - fall through, drain the
FIFO
            case 0x04:  // RX data available
                while (LSR & 0x01) {
                    uint16_t next = (rx_buf.head + 1) % RX_BUF_SIZE;
                    if (next != rx_buf.tail) {
```

```
                        rx_buf.buffer[rx_buf.head] = RBR_BYTE();
                        rx_buf.head = next;
                    } else {
                        (void)RBR;  // Discard if full (still pops the
FIFO)
                    }
                }
                break;

            case 0x02:  // TX empty
                while ((LSR & 0x20) && (tx_buf.tail != tx_buf.head)) {
                    THR = tx_buf.buffer[tx_buf.tail];
                    tx_buf.tail = (tx_buf.tail + 1) % TX_BUF_SIZE;
                }
                if (tx_buf.tail == tx_buf.head) {
                    IER &= ~0x02;  // Disable TX int
                }
                break;
        }
    }
}

size_t uart_write(const uint8_t *data, size_t len) {
    size_t written = 0;

    while (written < len) {
        uint16_t next = (tx_buf.head + 1) % TX_BUF_SIZE;
        if (next == tx_buf.tail) break;  // Buffer full

        tx_buf.buffer[tx_buf.head] = data[written++];
        tx_buf.head = next;
    }

    IER |= 0x02;  // Enable TX interrupt
    return written;
}

size_t uart_read(uint8_t *data, size_t max) {
    size_t count = 0;

    while ((rx_buf.tail != rx_buf.head) && (count < max)) {
        data[count++] = rx_buf.buffer[rx_buf.tail];
        rx_buf.tail = (rx_buf.tail + 1) % RX_BUF_SIZE;
    }
```

```
    return count;
}
```

26.2.3 Command Line Interface

```
#define CMD_BUF_SIZE 128

static char cmd_buf[CMD_BUF_SIZE];
static uint8_t cmd_pos = 0;

void cli_process_char(char c) {
    switch (c) {
        case '\r':
        case '\n':
            debug_puts("\r\n");
            cmd_buf[cmd_pos] = '\0';
            cli_execute(cmd_buf);
            cmd_pos = 0;
            debug_puts("> ");
            break;

        case '\b':
        case 0x7F:  // DEL
            if (cmd_pos > 0) {
                cmd_pos--;
                debug_puts("\b \b");
            }
            break;

        default:
            if (cmd_pos < CMD_BUF_SIZE - 1) {
                cmd_buf[cmd_pos++] = c;
                debug_putchar(c);
            }
            break;
    }
}

void cli_execute(const char *cmd) {
    if (strcmp(cmd, "help") == 0) {
        debug_puts("Available commands:\r\n");
        debug_puts("  help   - Show this help\r\n");
        debug_puts("  status - Show UART status\r\n");
    }
    else if (strcmp(cmd, "status") == 0) {
        char buf[64];
        sprintf(buf, "LSR: 0x%02X  MSR: 0x%02X\r\n", LSR, MSR);
        debug_puts(buf);
```

```
    }
    else if (cmd[0] != '\0') {
        debug_puts("Unknown command: ");
        debug_puts(cmd);
        debug_puts("\r\n");
    }
}
```

26.2.4 Loopback Test

```
bool uart_loopback_test(void) {
    uint8_t test_data[] = {0x00, 0x55, 0xAA, 0xFF};

    // Enable loopback mode
    MCR |= 0x10;

    // Clear FIFOs
    FCR = 0x07;

    // Send test data
    for (int i = 0; i < sizeof(test_data); i++) {
        THR = test_data[i];
    }

    // Wait for transmission
    while ((LSR & 0x40) == 0);

    // Short delay for internal loopback
    for (volatile int i = 0; i < 100; i++);

    // Read back and verify
    for (int i = 0; i < sizeof(test_data); i++) {
        if (!(LSR & 0x01)) {
            MCR &= ~0x10;
            return false;  // No data received
        }

        uint8_t received = RBR_BYTE();   // received byte is in RBR
bits [7:0]
        if (received != test_data[i]) {
            MCR &= ~0x10;
            return false;  // Data mismatch
        }
    }

    // Disable loopback
    MCR &= ~0x10;
```

```
    return true;
}
```

26.2.5 Flow Control (manual)

```
void uart_init_flow_control(void) {
    // Standard init - no DLAB toggle; DLL/DLM have their own offsets.
    DLL = 26; DLM = 0; LCR = 0x03;
    FCR = 0xC7;  // Enable FIFOs, trigger=14

    // Assert RTS (bit 1) and set OUT2 (bit 3) so the irq pin is
ungated.
    MCR = 0x0A;
}

// NOTE: this example does flow control by hand. Set MCR[5] (AFE)
instead
// and the hardware does it: CTS gates the start of a character and
RTS
// follows the RX FIFO level. MCR[1] must still be set for RTS to
assert.
```

26.3 Navigation

Back to: 00_overview.md - Programming Model Overview

Next Chapter: Chapter 5: Registers

---

27 APB UART 16550 — Register Map

27.1 Overview

This is the definitive register reference, and it documents what the RTL actually does — not what a 16550 datasheet says it should do. The address map is **additive**: every register has its own unique offset, AND the Divisor Latch Access Bit (LCR[7]) remaps 0x00/0x04 to DLL/DLM while it is set, as a classic 16550 does. DLL/DLM also keep their dedicated offsets (0x24/0x28), so both access styles work (RLB TASK-012, was RLB-013). Register offsets are byte offsets; the decode is strict, so an unmapped offset reads 0 with PSLVERR (see Address Calculation).

27.2 Functional Description

27.2.1 Register Summary

Offset	Register	Access	Reset	Description
0x00	RBR / THR	R / W	-	Receive Buffer (read) / Transmit Holding (write)
0x04	IER	RW	0x00	Interrupt Enable
0x08	IIR	RO	0x02	Interrupt Identification
0x0C	FCR	RW	0x00	FIFO Control (readable in this implementation)
0x10	LCR	RW	0x03	Line Control (8N1 at reset)
0x14	MCR	RW	0x00	Modem Control
0x18	LSR	RO, clear on read	0x60	Line Status
0x1C	MSR	RO, clear on read	0x00	Modem Status
0x20	SCR	RW	0x00	Scratch
0x24	DLL	RW	0x01	Divisor Latch LSB
0x28	DLM	RW	0x00	Divisor Latch MSB

Offsets 0x2C-0x3F are unused and acknowledge reads as zero.

27.2.2 RBR - Receiver Buffer Register (0x00, Read)

Bits	Name	Access	Description
7:0	DATA	RO	Received data byte
15:8	DATA_ALIAS	RO	The same byte again (see note)

Note: Reading offset 0x00 pops one entry from the RX FIFO and returns the received byte in bits **[7:0]**, as a standard 16550 does. THR shares the offset and is write-only: it has no storage and does not read back anywhere. Bits [15:8] mirror the same received byte and exist only for testbench helpers that predate the fix; do not write new software against them.

27.2.3 THR - Transmitter Holding Register (0x00, Write)

Bits	Name	Access	Description
7:0	DATA	WO	Data to transmit

Note: Writing offset 0x00 pushes data to the TX FIFO. The written value is also stored and is what appears in bits [7:0] of a subsequent RBR read (see RBR note); it is not otherwise software-visible.

27.2.4 IER - Interrupt Enable Register (0x04, R/W)

Bit	Name	Access	Reset	Description
0	ERBFI	RW	0	Enable Received Data Available Interrupt
1	ETBEI	RW	0	Enable Transmitter Holding Register Empty

Bit	Name	Access	Reset	Description
2	ELSI	RW	0	Enable Receiver Line Status Interrupt
3	EDSSI	RW	0	Enable Modem Status Interrupt
7:4	Reserved	RO	0	Reserved

Note: Each bit gates its own source. A disabled source still sets its status bit in LSR or MSR; it simply is not an interrupt, does not raise `irq` and is not reported by IIR. The `irq` pin is additionally gated by MCR.OUT2, which resets to 0. See `ch03_interfaces/04_interrupt.md`.

27.2.5 IIR - Interrupt Identification Register (0x08, Read Only)

Bits	Name	Access	Description
0	IPEND	RO	0=Interrupt pending, 1=No interrupt
3:1	IID	RO	Interrupt ID (see table below)
5:4	Reserved	RO	Reserved
7:6	FIFOEN	RO	11=FIFOs enabled, 00=disabled

27.2.5.1 Interrupt ID Encoding

IIR[0]=IPEND (0 = interrupt pending). IIR[3:1]=IID. IIR[3] (timeout) is always 0 in this implementation.

IIR[3:0]	Priority	Source	Clear Method
0110	1	Line Status	Read LSR to clear the error bits

IIR[3:0]	Priority	Source	Clear Method
0100	2	RX Data Available	Read RBR until DR clears
0010	3	THR Empty	Write THR (fills TX FIFO)
0000	4	Modem Status	Read MSR to clear the delta bits

Note: The **Character Timeout** interrupt (IIR = 0x0C, IIR[3]) is implemented. Reading IIR has **no** side effect; it does not clear the THR-empty condition. The THR-empty source is the level “TX FIFO empty” and clears only when the FIFO is refilled. Because interrupt enables are unimplemented (see IER), IIR reflects pending sources regardless of IER, and the `irq` pin is additionally gated by MCR.OUT2 (see MCR note).

27.2.6 FCR - FIFO Control Register (0x0C, R/W)

FCR is fully readable in this implementation (a standard 16550 FCR is write-only).

Bit	Name	Access	Description
0	FE	RW	FIFO Enable
1	RFR	RW	RX FIFO Reset (self-clearing)
2	TFR	RW	TX FIFO Reset (self-clearing)
3	DMS	RW	DMA Mode Select (stored; no effect - no DMA signals exist)
5:4	Reserved	RO	Reserved (read as 0)
7:6	RTL	RW	RX Trigger Level

27.2.6.1 RX Trigger Level

RTL[1:0]	Trigger Level
00	1 byte
01	4 bytes
10	8 bytes
11	14 bytes

27.2.7 LCR - Line Control Register (0x10, R/W)

Reset value is **0x03** (8 data bits, 1 stop bit, no parity - 8N1).

Bits	Name	Access	Reset	Description
1:0	WLS	RW	11	Word Length Select (reset = 8 bits)
2	STB	RW	0	Stop Bits (see note)
3	PEN	RW	0	Parity Enable
4	EPS	RW	0	Even Parity Select
5	SP	RW	0	Stick Parity
6	BC	RW	0	Break Control
7	DLAB	RW	0	Divisor Latch Access Bit: while set, 0x00 and 0x04 are DLL and DLM

Note (DLAB): LCR[7] remaps 0x00 to DLL and 0x04 to DLM, as a standard 16550 does. The dedicated offsets 0x24 and 0x28 reach the same latches whatever DLAB says, so either form works. **Note (STB):** STB=1 selects 2 stop bits for 6/7/8-bit words; 1.5 stop bits (for 5-bit words) is **not implemented** - a 5-bit word with STB=1 still produces 1 stop bit.

27.2.7.1 Word Length

WLS[1:0]	Data Bits
00	5
01	6
10	7
11	8

At every word length the received byte is right-justified and zero-filled: a 5-bit character reads in bits [4:0] with [7:5] clear. The transmitter sends bits [N-1:0] LSB-first, so TX and RX agree.

27.2.7.2 Parity Selection

PEN	EPS	SP	Parity
0	X	X	None
1	0	0	Odd
1	1	0	Even
1	0	1	Mark (1)
1	1	1	Space (0)

PEN	EPS	SP	Parity

27.2.8 MCR - Modem Control Register (0x14, R/W)

Bit	Name	Access	Reset	Description
0	DTR	RW	0	Data Terminal Ready
1	RTS	RW	0	Request To Send
2	OUT1	RW	0	User Output 1 (active low)
3	OUT2	RW	0	User Output 2 / interrupt gate (active low) - see note
4	LOOP	RW	0	Loopback Mode
5	AFE	RW	0	Auto Flow Control Enable
7:6	Reserved	RO	0	Reserved (read as 0)

Note: Bit 5 (AFE) gates the transmitter with CTS and drives RTS from the RX FIFO level; MCR[1] must still be set for RTS to assert. Writes above bit 5 are dropped. There is no CTS-gated transmit and RTS is not auto-driven by RX FIFO level. Additionally, OUT2 gates the `irq` output pin: `irq` can assert only when MCR.OUT2 = 1. With the reset value MCR = 0x00, the `irq` pin is masked; software must set MCR.OUT2 to route interrupts to the pin.

27.2.9 LSR - Line Status Register (0x18, Read; error bits clear on read)

Bit	Name	Access	Description
0	DR	RO	Data Ready
1	OE	RO	Overrun Error (clears when the register is read)
2	PE	RO	Parity Error (clears when the register is read)
3	FE	RO	Framing Error (clears when the register is read)
4	BI	RO	Break Interrupt (clears when the register is read)

Bit	Name	Access	Description
5	THRE	RO	Transmitter Holding Register Empty (TX FIFO empty)
6	TEMT	RO	Transmitter Empty (FIFO and shift register empty)
7	FIFOERR	RO	At least one parity error, framing error or break indication in the RX FIFO (FIFO mode only)

Note: The error bits [4:1] **clear when LSR is read**, as PC16550D specifies. Any read clears them, including one that was only looking at DR, so a polling loop consumes the error it is polling for - capture the value the poll returned rather than reading LSR twice. PE, FE and BI are the tags of the character the CPU is being handed, not a running OR over everything received; OE is a port-level flag. FIFOERR (bit 7) aggregates over the whole FIFO and is defined only in FIFO mode.

27.2.10 MSR - Modem Status Register (0x1C, Read; delta bits clear on read)

Bit	Name	Access	Description
0	DCTS	RO	Delta Clear To Send (clears when the register is read)
1	DDSR	RO	Delta Data Set Ready (clears when the register is read)
2	TERI	RO	Trailing Edge Ring Indicator (clears when the register is read)
3	DDCD	RO	Delta Data Carrier Detect (clears when the register is read)
4	CTS	RO	Clear To Send
5	DSR	RO	Data Set Ready
6	RI	RO	Ring Indicator
7	DCD	RO	Data Carrier Detect

Note: The delta bits [3:0] **clear when MSR is read**, as PC16550D specifies, and the modem-status interrupt deasserts with them. As with LSR, any read clears them, so work from the value the read returned rather than reading MSR twice.

27.2.11 SCR - Scratch Register (0x20, R/W)

Bits	Name	Access	Reset	Description
7:0	DATA	RW	0x00	General-purpose storage

27.2.12 DLL - Divisor Latch LSB (0x24, R/W)

Bits	Name	Access	Reset	Description
7:0	DLL	RW	0x01	Baud rate divisor low byte

Note: Reset value is **0x01** (not 0x00). Accessed directly at 0x24 - no DLAB toggle required.

27.2.13 DLM - Divisor Latch MSB (0x28, R/W)

Bits	Name	Access	Reset	Description
7:0	DLM	RW	0x00	Baud rate divisor high byte

Note: Accessed directly at 0x28 - no DLAB toggle required.

27.2.14 Address Calculation

```
Register_Address = BASE_ADDR + Register_Offset

Where:
  BASE_ADDR       = RLB/UART base address (system-integration
dependent)
  Register_Offset = value from the Register Summary table above

Only the eleven mapped offsets decode (paddr[11:6] must be zero);
```

```
every
other address in the window reads 0 and returns PSLVERR, so nothing
aliases.

Example (BASE_ADDR = 0xFEC08000):
  LSR = 0xFEC08000 + 0x18 = 0xFEC08018
```

27.3 Implemented for RLB TASK-012 (was RLB-013)

The five 16550 features this block used to leave out are now built:

Feature	Behaviour
Character-timeout interrupt	Four character times with the RX FIFO non-empty and no activity; IIR reads 0x0C, sharing the received-data slot and distinguished by IIR[3]. Gated by IER[0], FIFO mode only
Auto flow control (AFE)	MCR[5]. CTS gates the start of a character; RTS follows the RX FIFO level, deasserting at the trigger and reasserting once read below it. MCR[1] must still be set for RTS to assert at all
1.5 stop bits	LCR[2] with a 5-bit character sends a half-length second stop bit: 7.5 bit times against 7
DLAB remapping	While LCR[7] is set, 0x00 and 0x04 are the divisor latches. The flat offsets 0x24 and 0x28 keep working, so both forms reach the same latches
DMA mode select	FCR[3] picks the handshake on the `rxrdy_n` / `txrdy_n` pins: mode 0 per character, mode 1 per block

Two items still constrain integration rather than function. The block instantiates no reset synchronizer, so `presetn` and `uart_rstn` must arrive already synchronized, and resetting one domain with a transfer in flight corrupts the asynchronous FIFOs - quiesce the bus first, or assert both together. An illegal parameter fails in simulation at time 0 rather than at elaboration, because the guards are `initial` blocks and synthesis ignores them.